ShieldFL: Mitigating Model Poisoning Attacks in Privacy-Preserving Federated Learning

Zhuoran Ma, Jianfeng Ma, *Member, IEEE*, Yinbin Miao, *Member, IEEE*, Yingjiu Li, *Member, IEEE*, and Robert H. Deng, *Fellow, IEEE*

***Abstract*— Privacy-Preserving Federated Learning (PPFL) is an emerging secure distributed learning paradigm that aggregates user-trained local gradients into a federated model through a cryptographic protocol. Unfortunately, PPFL is vulnerable to model poisoning attacks launched by a Byzantine adversary, who crafts malicious local gradients to harm the accuracy of the federated model. To resist model poisoning attacks, existing defense strategies focus on identifying suspicious local gradients over plaintexts. However, the Byzantine adversary submits encrypted poisonous gradients to circumvent existing defense strategies in PPFL, resulting in encrypted model poisoning. To address the issue, in this paper we design a privacy-preserving defense strategy using two-trapdoor homomorphic encryption (referred to as ShieldFL), which can resist encrypted model poisoning without compromising privacy in PPFL. Specially, we first present the secure cosine similarity method aiming to measure the distance between two encrypted gradients. Then, we propose the Byzantine-tolerance aggregation using cosine similarity, which can achieve robustness for both Independently Identically Distribution (IID) and non-IID data. Extensive evaluations on three benchmark datasets (*i.e.,* MNIST, KDDCup99, and Amazon) show that ShieldFL outperforms existing defense strategies. Especially, ShieldFL can achieve 30% − 80% accuracy improvement to defend two state-of-the-art model poisoning attacks in both non-IID and IID settings.**

Manuscript received November 7, 2021; revised February 14, 2022 and April 3, 2022; accepted April 4, 2022. Date of publication April 22, 2022; date of current version May 6, 2022. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB3101100; in part by the Foundation for Innovative Research Groups of the National Natural Science Foundation of China under Grant 62121001; in part by the National Natural Science Foundation of China under Grant 62072361, Grant U1804263, and Grant 62072352; in part by the Key Research and Development Program of Shaanxi Province under Grant 2019ZDLGY12-04 and Grant 2020ZDLGY09-06; in part by the Fundamental Research Funds for the Central Universities under Grant JB211505; in part by the Guangxi Key Laboratory of Cryptography and Information Security under Grant GCIS201917; and in part by the Guangxi Key Laboratory of Trusted Software under Grant KX202028. The work of Yingjiu Li was supported in part by the Ripple University Blockchain Research Initiative. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. George Theodorakopoulos. *(Corresponding author: Yinbin Miao.)*

Zhuoran Ma is with the State Key Laboratory of Integrated Services Network, School of Cyber Engineering, Xidian University, Xi'an 710071, China, and also with the School of Information Systems, Singapore Management University, Singapore 188065 (e-mail: emmazhr@163.com).

Jianfeng Ma and Yinbin Miao are with the State Key Laboratory of Integrated Services Network, School of Cyber Engineering, Xidian University, Xi'an 710071, China (e-mail: jfma@mail.xidian.edu.cn; ybmiao@xidian.edu.cn).

Yingjiu Li is with the Computer and Information Science Department, University of Oregon, Eugene, OR 97403 USA (e-mail: yingjiul@uoregon.edu).

Robert H. Deng is with the School of Information Systems, Singapore Management University, Singapore 188065 (e-mail: robertdeng@smu.edu.sg).

Digital Object Identifier 10.1109/TIFS.2022.3169918

***Index Terms*— Privacy-preserving, homomorphic encryption, defense strategy, model poisoning attack, federated learning.**

I. Introduction

FEDERATED Learning (FL) [1], [2] is an advanced distributed machine learning paradigm, which trains a federated model with downstream aggregations based on locally-trained gradients instead of user-held data. In FL, it is possible to discover sensitive data distributions from user-provided model gradients, resulting in privacy leakage [3], [4]. Therefore, one workaround in Privacy-Preserving FL (PPFL) adopts cryptographic primitives for data encryption and secure aggregation, in order to minimize privacy disclosure [5]. In a typical PPFL scheme, the server implements secure aggregation on encrypted local updates with strong privacy guarantees.

With the potential benefit, PPFL has attracted widespread attentions. There are, however, a number of challenges associated with PPFL [6], [7]. Due to the invisibility of local training, PPFL is inherently vulnerable to model poisoning attacks that significantly hinder the performance of a federated model [8], [9]. Model poisoning attack is launched by a Byzantine adversary, who manipulates an arbitrary proportion of malicious users to deviate from a correct trend by submitting poisonous local model updates. With the aim of influencing the learning outcome and misinforming decision-making, model poisoning attacks can cause the accuracy of a federated model to drop sharply, and thus degrade the FL performance. Taking Fig. 1 as an example, certain malicious users provide poisonous local gradients, which are diverged from benign local gradients in order to manipulate the convergence direction of the aggregated gradient. The adversarial objective of malicious users is to cause the federated model to yield attacker-chosen target labels for specific samples (called *targeted attacks*) [8], [10], or to misclassify all testing samples indiscriminately (called *untargeted attacks*) [11].

To defend against model poisoning attacks in PPFL, existing works that have explored several defense measures still confront the following challenges. *i) Failure in encrypted model poisoning*. The model poison attack is covert and hard to be observed or detected in PPFL. Especially, PPFL exploits cryptographic primitives to provide privacy protection, in the sense that the original information should not be leaked from encrypted local gradients. The ciphertext view of local training provides concealment for poisonous updates, which

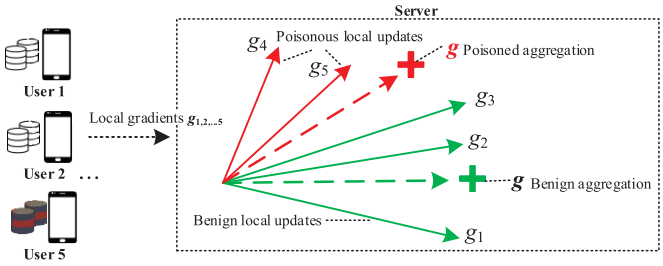


Fig. 1. Example of model poisoning attacks in FL. There are 5 users, g_1, g_2, g_3 are local benign gradients, and g_4, g_5 are local poisonous gradients.

hides the original information using encrypted gradients. PPFL increases the difficulty of detecting such attacks. As a result of the dilemma, current defense strategies cannot prevent encrypted model poisoning if malicious gradients cannot be identified without learning statistical distributions in PPFL. Thus, encrypted poisonous gradients may easily circumvent most existing defense strategies in PPFL. *ii) Privacy leakage in PPFL.* Existing defense strategies [12], [13] may compromise the privacy in PPFL. If defense strategies have to access local gradients in defending against model poisoning attacks, then local gradients are inevitably leaked, which causes privacy leakage in PPFL. *iii) Unpredictability with heterogeneous data*. PPFL is trained with heterogeneous data, including Independently Identically Distribution (IID) and non-IID data. Heterogeneous data (especially non-IID data) introduce non-negligible local biases in PPFL. Benign gradients trained on local IID data typically follow a similar distribution, among which poisonous gradients may be easily detected with anomalies. Differently, benign gradients trained on non-IID data are divergent, among which current defense strategies are difficult to detect poisonous gradients. Since PPFL cannot access local data, the distributions of local data and local gradients are unknown. Therefore, it is difficult to design a privacy-preserving defense strategy for heterogeneous data settings in PPFL.

The above issues make it challenging to design an effective defense strategy against model poisoning attacks in PPFL. First, encrypted poisonous gradients offer covertness in model poisoning. As a potential technique for implementing a defense approach on encrypted gradients, Homomorphic Encryption (HE) is introduced in PPFL, but incurs huge computation overheads. It is difficult to design an efficient defense countermeasure since existing defense strategies involve complicated HE computations with unaffordable computation costs in PPFL. Second, in the defense phase, the intermediate parameters learnt by the server may be vulnerable to reconstruction attacks [14] and inference attacks [4], which may leak the privacy of local data in PPFL. It is also important to guarantee the security of the secret key, since key disclosure can cause severe privacy problems in PPFL. Third, heterogeneous data scenarios in PPFL may cause quite distinct distributions of local gradients, making it unpredictable to prevent poisonous gradients. It is prone to misjudge a benign gradient with a distinct variation as the poisonous one.

To solve these challenges, we propose ShieldFL, a privacy-preserving defense strategy against model poisoning attacks in PPFL, which is based on two-trapdoor HE mechanism to prevent key disclosure and data leakage, ensuring data privacy. In ShieldFL, we first present a privacy-preserving defense strategy using cosine similarity to resist encrypted model poisoning in PPFL. Then, we design a Byzantine-tolerance aggregation mechanism for heterogeneous data scenarios with robustness. The main contributions are summarized as follows.

- We propose a privacy-preserving defense strategy based on two-trapdoor HE to resist encrypted model poisoning, which can identify encrypted malicious gradients based on cosine similarity with privacy guarantees and high efficiency.
- We design a Byzantine-tolerance aggregation mechanism to ensure robustness in ShieldFL, which supports heterogeneous data scenarios including IID and non-IID data, aiming to mitigate an arbitrary proportion (less than 50%) of malicious users.
- We evaluate ShieldFL on three real-world datasets against two representative model poisoning attacks: targeted attacks and untargeted attacks. The experiments show that ShieldFL can identify poisonous gradients with high accuracy and efficiency.

The rest of the paper is organized as follows. In Section II, we introduce the related literatures. A set of machine learning and security primitives that are utilized in ShieldFL are provided in Section III. We define the system model, threat model and design goals in Section IV. The ShieldFL framework is explained in details in Section V. Section VI and Section VII discuss the theoretical and performance analysis. We conclude the paper in Section VIII.

II. Related Work

In this section, we introduce model poisoning attacks and the latest defense countermeasures in PPFL.

PPFL, as a kind of privacy-preserving machine learning paradigms, has been widely investigated in recent years. Google [2] first proposed PPFL, which is based on Stochastic Gradient Descent (SGD). Multiple users locally train individual gradients and submit local model updates to a semi-honest server that then implements the aggregation. Since inference attacks may disclose the data distribution over model updates [15], PPFL adopts cryptographic techniques to protect transmitted model updates without privacy leakage.

Even though existing PPFL systems provide security protection to some extent, PPFL still has a potential to suffer from model poisoning attacks with poisonous updates or corrupted local gradients [8], [16], [17]. The model poisoning attack was first presented in [11], where the adversary corrupts locally-trained model parameters to poison the aggregation. By injecting encrypted poisonous updates instead of injecting poisonous samples, model poisoning attacks are more concealable and powerful than traditional data poisoning attacks [18], [19], which seriously hamper the development of PPFL. Model poisoning attacks can be classified into two categories based on the adversarial goal, namely *untargeted attack* and *targeted attack*. Untargeted attacks [11] aim to disrupt the availability of the learned model in the testing phase with the high error

rate, resulting in denial-of-service. Targeted attacks [8], [10] aim to corrupt the integrity of the learned model, which can cause mis-predictions on testing samples of the specific labels at the test time, while preserving the correct predictions on other testing samples.

A number of solutions have been proposed against model poisoning attacks in FL. The median-based defense strategy [12] proposes the median statistics against Byzantine failures, which computes the median of local gradients as the global update in FL. Krum [13] adopts the Euclidean distance to determine outliers. The gradient with the closest distance to its neighboring gradients is selected as the global gradient. However, these schemes [12], [13] ignore a fundamental issue in FL, *i.e.,* FL needs to aggregate local updates for the global convergence. To address the problem, Bulyan [20] selects certain local gradients using Krum and then aggregates selected local gradients for the global gradient. Auror [21] uses k-means to cluster local model updates and identify the outliers in FL. Sybils [17] uses cosine similarity to identify the closest gradients as the outliers against model poisoning in non-IID, meaning malicious gradients have similar variations that are distinct from benign gradients. Unfortunately, the practicality of these solutions is limited without a Byzantine-tolerance mechanism that ensures the applicability of a defense strategy across heterogeneous data in FL. As local updates trained on non-IID data have relatively distinct distributions [22], [23], these schemes in [20] and [21] may have a high probability in misjudging benign local updates trained on non-IID data as outliers. Sybils [17] also demonstrates a significant accuracy drop in an IID scenario, where IID gradients with similar distributions are easily regarded as malicious gradients. On the other hand, FL-trust [24] is a Byzantine-tolerance mechanism against model poisoning based on the assumption of a clean validation dataset held by the server, which identifies the outliers by measuring the cosine similarity of each local gradient to the benign gradient trained on the validation data. With the same assumption, the trimmed-means method [11] is also proposed with a validation dataset. Unfortunately, these schemes [11], [24] are not applicable in PPFL setting, since the server in PPFL is prohibited from collecting and operating training and validation data directly in compliance with privacy policies [7], [25]. With respect to privacy, Liu *et al.* [26] proposed the Privacy-Enhanced FL (PEFL) against poisoning attacks with IID data, which uses pearson correlation coefficient to identify the outliers. It adopts the two-server model for secure computation, in which a trusted server holds the secret key to decrypt intermediate encrypted parameters. This defense strategy takes the strong security assumption that the confidentiality of the secret key cannot be compromised. Once the server is corrupted by the adversary, it is easy to leak the secret key, resulting in privacy leakage.

Therefore, it is a major challenge to prevent encrypted model poisoning attacks in both IID and non-IID data scenarios. To the best of our knowledge (see TABLE I), existing strategies are not effective in defending encrypted model poisoning attacks in PPFL, due to the limited ability of existing defense solutions to distinguish outliers among encrypted gradients.

TABLE I
A COMPARATIVE SUMMARY OF EXISTING DEFENSE STRATEGIES

Approach	Setting	Defense Strategy	Privacy	Byzantine Tolerance
Median [12]	IID	Median statistics	○	○
Krum [13]	IID	Euclidean distance	○	○
FL-trust [24]	IID	Cosine similarity, Validation data	○	●
Trimmed-mean [11]	Both	Validation data	◐	◐
Bulyan [20]	IID	Euclidean distance	○	◐
Auror [21]	IID	k-means	○	○
Sybils [17]	Non-IID	Cosine similarity	○	●
PEFL [26]	IID	Pearson correlation coefficient	◐	●
Our work	**Both**	Cosine similarity	●	●

Notes. ○ denotes the approach does not consider or solve. ◐ denotes the approach does not resolve completely, ● denotes the approach can solve.

III. TECHNICAL BACKGROUND

This section first presents two-trapdoor HE, and then introduces the FL framework.

A. Two-Trapdoor Homomorphic Encryption

Two-trapdoor homomorphic encryption scheme is based on the Paillier cryptosystem, which provides the following algorithms. The details are shown in [27], [28].

- $\mathsf{KeyGen}(\varepsilon) \rightarrow (pk, sk)$: Given the security parameter ε, distinct odd primes p, q are generated, where $|p| = |q| = \varepsilon$, $N = pq$. The public key $pk = (N, (1+N))$ and secret key $sk = \lambda = lcm(p-1, q-1)$ are yielded.
- $\mathsf{Enc}_{pk}(x) \rightarrow [\![x]\!]$: Given a plaintext $x \in \mathbb{Z}_N$, it is encrypted with pk such that

$$[\![x]\!] = (1+N)^x \cdot r^N \mod N^2, \quad r \in \mathbb{Z}_N^*. \tag{1}$$

- $\mathsf{KeySplit}(sk) \rightarrow (sk_1, sk_2)$: The secret key $sk = \lambda$ is randomly divided into two secret key shares sk_1 and sk_2 satisfying

$$\sum_{i=1}^{2} sk_i \equiv 0 \mod \lambda, \quad \sum_{i=1}^{2} sk_i \equiv 1 \mod N. \tag{2}$$

- $\mathsf{PartDec}_{sk_i}([\![x]\!]) \rightarrow [x]_i$: Given an encrypted data $[\![x]\!]$ and a secret key share sk_i, it yields the corresponding decryption share $[x]_i$ with sk_i such that

$$[x]_i = [\![x]\!]^{sk_i} \mod N^2. \tag{3}$$

- $\mathsf{FullDec}([x]_1, [x]_2) \rightarrow x$: Given the tuple of decryption shares $([x]_1, [x]_2)$, the plaintext x is decrypted as

$$x = \frac{(\prod_{i=1}^{2} [x]_i \mod N^2) - 1}{N} \mod N. \tag{4}$$

To decrypt an encrypted number, both the $\mathsf{PartDec}$ and $\mathsf{FullDec}$ algorithms must be used.

Suppose that there are two semi-honest parties (*i.e.,* Alice and Bob), two-trapdoor HE designs the secure multiplication over ciphertexts based on additive homomorphism. Given two ciphertexts $[\![x_1]\!]$, $[\![x_2]\!]$ and an integer $k \in \mathbb{Z}_N$, two-trapdoor HE satisfies the additive homomorphism such that

$$[\![x_1]\!] \cdot [\![x_2]\!] = [\![x_1 + x_2]\!], \quad [\![x_1]\!]^k = [\![k \cdot x_1]\!]. \tag{5}$$

Since HE is operated with integers, a float-point number is processed as an integer before Enc. We adopt the approximation method Appr to convert a float-point number x to an integer $x' \leftarrow \mathsf{Appr}(x)$ using a precision degree deg such that

$$\mathsf{Appr}(x) = \lfloor x \cdot deg \rceil. \tag{6}$$

Note that the precision degree deg is expanded after a multiplication operation as

$$x'_1 * x'_2 = \lfloor x_1 * deg \rceil * \lfloor x_2 * deg \rceil \approx x_1 * x_2 * deg^2.$$

To keep a precision consistent, a secure truncation function HE.Trun is required in HE.Mul. The specified process is shown in [28].

- $\mathsf{HE.Mul}(\llbracket x'_1 \rrbracket, \llbracket x'_2 \rrbracket) \rightarrow \llbracket x_{mul} \rrbracket$: Given two ciphertexts $\llbracket x'_1 \rrbracket$ and $\llbracket x'_2 \rrbracket$, the multiplication result $\llbracket x_{mul} \rrbracket$ is yielded with the interaction between Alice and Bob, where $x_{mul} = x'_1 \times x'_2 = x_1 * x_2 * deg^2$.
- $\mathsf{HE.Trun}(\llbracket x_{mul} \rrbracket, deg) \rightarrow \llbracket x \rrbracket$: To reduce the noise deg involved in $\llbracket x_{mul} \rrbracket$ after a HE.Mul, the truncated result $\llbracket x \rrbracket$ is yielded between Alice and Bob, where $x = \lfloor \frac{x_{mul}}{deg} \rceil = x_1 * x_2 * deg$.

B. Federated Learning

Assume that heterogeneous training data are distributed among n users. FL is trained with Stochastic Gradient Descent (SGD) after multiple training iterations.

- *Local training*. In the $t-$th training iteration, each user trains individual local gradient $g_i^{(t)}$ using the local model W_i over local data D_i ($i \in [1, n]$). To minimize the loss function $\mathcal{L}_i(W_i, D_i)$, $g_i^{(t)}$ is obtained for a local optimization such that

$$\text{argmin}_{W_i} \ \mathcal{L}_i(W_i, D_i), \quad g_i^{(t)} = \frac{\partial \mathcal{L}_i}{\partial W_i}. \tag{7}$$

  The local gradient $g_i^{(t)}$ is sent to the server.
- *Aggregation*. The server aggregates local gradients $g_i^{(t)}$ to obtain the global gradient $g^{(t)}$ as

$$g^{(t)} = \frac{1}{n} \sum\nolimits_{i=1}^{n} g_i^{(t)}, \tag{8}$$

- *Weight updates*. Each user updates the local model W_i after receiving the global gradient $g^{(t)}$ such that

$$W_i{}^{(t)} := W_i{}^{(t-1)} - lr^{(t)} g^{(t)}, \tag{9}$$

  where $lr^{(t)}$ is the learning rate in the t-th iteration.

For PPFL, each local gradient is encrypted as $\llbracket g_i \rrbracket$ using the public key pk and the aggregation process is operated with encrypted local gradients involving secure computations. Here, we employ two-trapdoor HE to achieve PPFL.

IV. Problem Formulation

In this section, we formalize the system model, adversarial model and design goals of ShieldFL, respectively.

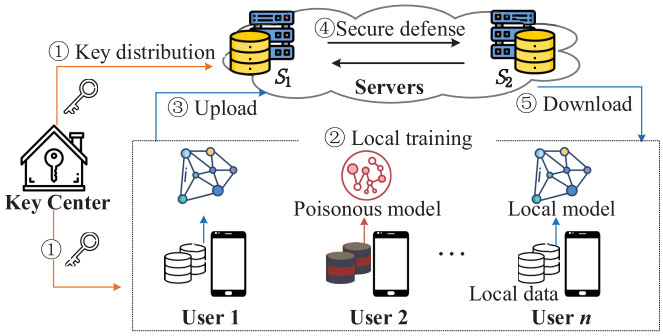


Fig. 2. System model. Such as "User 1" and "User n" are benign, and "User 2" is malicious.

A. System Model

As depicted in Fig. 2, the system model consists of a key center KC, two cloud servers $\mathcal{S}_1$ and $\mathcal{S}_2$, and n users $\mathcal{U}_{i\in[1,n]}$. The concrete roles in ShieldFL are elaborated as follows:

- *Key center*. KC generates and distributes system keys (*i.e.,* public key and secret key shares) for $\mathcal{S}_1$, $\mathcal{S}_2$ and $\mathcal{U}$ (Step ①).
- *Users*. In the t-th training iteration, $\mathcal{U}_i$ performs the local training over individual dataset D_i, and encrypts local gradients $g_i^{(t)}$ with the public key before uploading them to $\mathcal{S}_1$ (Step ②-③). Then, the aggregated global gradient $g^{(t)}$ is downloaded for local tuning (Step ⑤).
- *Servers*. $\mathcal{S}_1$ and $\mathcal{S}_2$ interact to implement a privacy-preserving defense countermeasure using secure protocols to resist model poisoning attacks in PPFL, which is executed with encrypted local gradients $\llbracket g_i^{(t)} \rrbracket$ for the aggregated gradient $\llbracket g^{(t)} \rrbracket$ (Step ④).

B. Adversarial Model

KC is a fully trusted party. $\mathcal{S}_1$ and $\mathcal{S}_2$ are two honest-but-curious and non-colluding entities, which are honest to follow the pre-defined protocol but curious to compromise users' privacy. The non-collusion of the twin-server model has been formalized in prior work [29]. Since most companies with cloud servers are well-established, the collusion will harm their individual interests and credibility. In ShieldFL, $\mathcal{U}$ is curious to learn other users' data, and we consider two types of users, *i.e.,* benign users and malicious users. We give the definitions of *benign users* and *malicious users* as follows.

Definition 1 (Benign Users): A user $\mathcal{U}_i$ is benign if and only if $\mathcal{U}_i$ honestly submits local gradients g_i, where g_i is an unbiased estimator of the true gradient trained on its local dataset D_i.

Definition 2 (Malicious Users): A user $\mathcal{U}_i^*$ is malicious if and only if $\mathcal{U}_i^*$ is manipulated by a Byzantine adversary $\mathcal{A}$ who launches a model poisoning attack by submitting encrypted poisonous gradients $\llbracket g_i^* \rrbracket$.

We define the model poisoning attack scenario.

Definition 3 (Model Poisoning Attack): The model poisoning attack launched by the Byzantine adversary $\mathcal{A}$ is described as follows [11].

- *Adversarial goals*. $\mathcal{A}$ corrupts the local training process to decrease the accuracy of the federated model, wherein

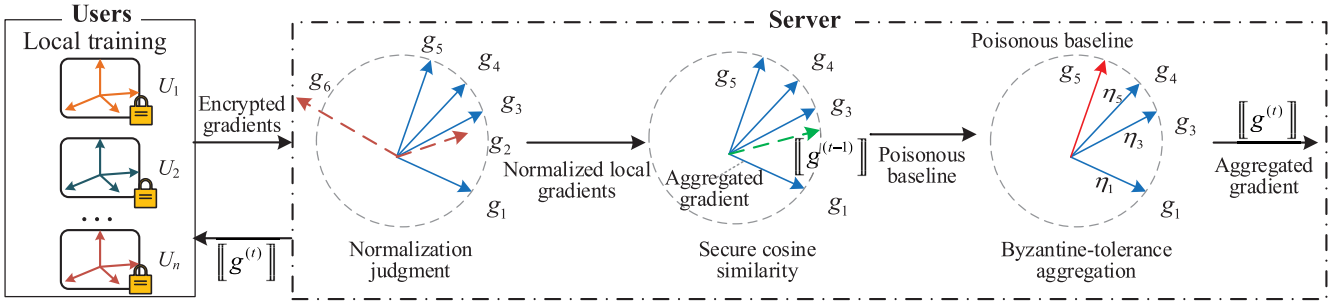


Fig. 3. Overview of ShieldFL.

local gradients can be maliciously crafted to affect the federated model.

- *Adversarial knowledge and capability*. $\mathcal{A}$ controls one or more malicious users $\mathcal{U}^*$, where $\mathcal{U}^*$ obtains local feature knowledge, local model W_i, local instance knowledge D, and training process.
- *Model poisoning strategy*. $\mathcal{A}$ aims to solve the following optimization problem as

$$\text{argmax}_{W_{i\in[1,n]}} H^T(W - W^*), \tag{10}$$

where H denotes a column vector of changing directions of the federated model, W is a before-attack federated model over benign updates g_i, and W^* is a poisoned federated model over poisonous local updates g_i^*.

TABLE II
NOTATION DESCRIPTIONS

Notations	Descriptions	Notations	Descriptions
$\mathbb{Z}^*_{N^2}$	Ciphertext space	$\mathbb{Z}_N$	Plaintext space
pk, sk	Public/secret key	sk_i	Secret key share
$[\![x]\!]$	Ciphertext	$[x]_i$	Decryption share with sk_i
g_i	Local gradient	g	Aggregated gradient
g_i^*	Poisonous gradient	W^*	Poisoned model
$W_i^{(t)}$	Local model	$W^{(t)}$	Global model
Appr	Approximation function	deg	Precision degree
HE.Mul	Secure multiplication	SecJudge	Normalization judgment
SecCos	Secure cosine similarity	cos	cosine similarity value
Acc	Overall accuracy	Acc_{source}	Source accuracy
AI	Improvement of Acc	ASR	Attack success rate
AI_{source}	Improvement of Acc_{source}	η_i	Confidence of $\mathcal{U}_i$

C. Design Goals

Under the above adversarial model, ShieldFL is designed to achieve two design goals.

Goal 1 (Defense Countermeasure): ShieldFL should resist encrypted model poisoning attacks with an arbitrary size of malicious users $\mathcal{U}^*$ under heterogeneous data settings ($|\mathcal{U}^*| < |\mathcal{U}|/2$), including IID and non-IID settings. The proposed defense countermeasure is reliable in PPFL.

Goal 2 (Privacy Preservation): ShieldFL should guarantee that the defense process is confidential. All private data of users' inputs and final federated model should not be leaked to other parties (such as $\mathcal{S}_1$, $\mathcal{S}_2$ and malicious users $\mathcal{U}^*$).

V. ShieldFL Framework

In this section, we first describe the technical overview, then demonstrate the concrete construction of ShieldFL. The notations are described in TABLE II.

A. Technical Overview

Fig. 3 shows the overview of the privacy-preserving defense strategy, which uses the cosine similarity technique to resist poisonous gradients among encrypted gradients on the server side. In ShieldFL, the local model W_i and the global model W are represented as vectors. As the locally-submitted gradients are multiple dimensional vectors, their outliers can be identified by using the cosine similarity between local gradients and the aggregated gradient. Compared with existing defense strategies using cosine similarity [17], [24], ShileldFL enjoys the advantages of supporting heterogeneous settings (for both IID and non-IID data) and guaranteeing privacy. To support heterogeneous data settings, ShieldFL designs the Byzantine-tolerance aggregation against model poisoning attacks, which prevents misjudgements on outliers. It resolves the limitation in [17], [24] of handling both IID and non-IID data, and avoids the potential risk of violating data privacy caused by collecting validation data on the server side [24]. For privacy preservations, ShiledFL relies on two-trapdoor HE to encrypt local gradients on user side and enable the privacy-preserving defense strategy to be performed on the server side without leaking any information about users' data.

In the t-th training iteration, if a local gradient $[\![g_i^{(t)}]\!]$ has a large magnitude without being normalized, it may be the poisonous one to mislead the aggregated gradient. Thus, we propose a SecJudge function to implement *normalization judgment*, which determines whether an encrypted local gradient $[\![g_i^{(t)}]\!]$ submitted to $\mathcal{S}_1$ is normalized. Since local gradients $[\![g_i^{(t)}]\!]$ are encrypted data, we design the SecCos function to implement the *secure cosine similarity* between two encrypted gradients involving HE.Mul. If a local gradient is dissimilar from an aggregated gradient, it is more likely to be poisonous from being detected and isolated (See Fig.4(a)). Unfortunately, FL is constructed in heterogeneous data settings, with which we observe the following complicated cases. Fig.4(b) depicts that one local gradient trained by a benign user has the lowest cosine similarity with the aggregated gradient. It is thus difficult to identify poisonous gradients precisely using

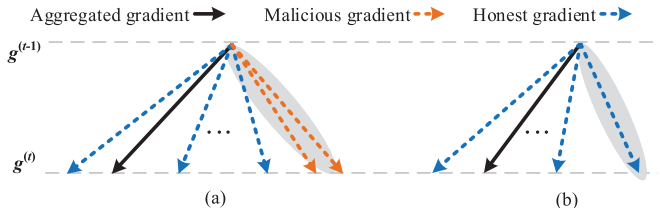


Fig. 4. Examples of malicious gradients in ShieldFL.

the cosine similarity. In this way, a local gradient trained by a benign user is easily mistaken for a poisonous one. To avoid the misjudgment of a local gradient trained by a benign user, we design the *Byzantine-tolerance aggregation* mechanism, which uses the confidence η_i ($i \in [1, n]$) instead of discarding abnormal gradients.

B. Construction of ShieldFL

ShieldFL consists of three processes, namely setup, local training, and privacy-preserving defense strategy. We describe the detailed processes as follows.

1) Setup: In the setup process, KC implements key distribution, and $\mathcal{S}_1$ distributes the initial model to $\mathcal{U}$. The details are shown in Algorithm 1.

Algorithm 1 Setup

Input: Security parameter ε.
Output: System keys $pk, sk_1, sk_2, sk_{u_i}, sk_{s_i}$.

/*Key distribution*/
KC generates $(pk, sk) \leftarrow \mathsf{KeyGen}(1^{\varepsilon})$;
KC implements $(sk_1, sk_2) \leftarrow \mathsf{KeySplit}(sk)$;
for $\mathcal{U}_{i\in[1,n]} \in \mathcal{U}$ **do**
  KC implements $(sk_{u_i}, sk_{s_i}) \leftarrow \mathsf{KeySplit}(sk)$;
return KC broadcasts pk and distributes sk_{s_i}, sk_1 to $\mathcal{S}_1$, sk_2 to $\mathcal{S}_2$, and sk_{u_i} to $\mathcal{U}_{i\in[1,n]}$;
/*Model distribution*/
for $\mathcal{U}_{i\in[1,n]} \in \mathcal{U}$ **do**
  $\mathcal{U}_i$ downloads $[\![W^{(0)}]\!]$ and partial decryption $[W^{(0)}]_{sk_{s_i}} \leftarrow \mathsf{PartDec}_{sk_{s_i}}([\![W^{(0)}]\!])$ from $\mathcal{S}_1$;
  $\mathcal{U}_i$ decrypts and initializes $W^{(0)}$ using $\mathsf{FullDec}$.

a) Key distribution: KC generates and distributes system keys for $\mathcal{S}_1$, $\mathcal{S}_2$ and $\mathcal{U}$. KC splits the secret key sk into sk_1 and sk_2, which are sent to $\mathcal{S}_1$ and $\mathcal{S}_2$, respectively. Then, KC splits sk into sk_{s_i} and sk_{u_i} for $\mathcal{S}_1$ and $\mathcal{U}_i$, respectively ($i \in [1, n]$). ShieldFL relies on the key split algorithm $\mathsf{KeySplit}$ to distribute secret key shares among $\mathcal{S}_1$, $\mathcal{S}_2$ and $\mathcal{U}$ to prevent key disclosure.

Remark: To protect data privacy, the single-key HE could be exploited for a single server [5]. In such case, KC generates a public-secret key pair (pk, sk), where sk is sent to each user $\mathcal{U}_i$, pk is broadcasted. The single server $\mathcal{S}$ implements secure computations in the defense process instead of the two servers in ShieldFL. However, since $\mathcal{U}_i$ holds the secret key sk, if a malicious user leaks his/her secret key sk, the adversary $\mathcal{A}$ could decrypt all encrypted information thus violating ShieldFL security. Besides, the twin-server secure computation scheme based on the single-key HE has been proposed in [26]. To implement secure computation, a single server $\mathcal{S}_1$ or $\mathcal{S}_2$ holds a secret key sk that can decrypt ciphertexts directly. It is also challenging to guarantee the confidentiality of sk as $\mathcal{S}_{i\in[1,2]}$ is not fully-trusted. Beyond that, their secure protocols not only output a ciphertext of a plaintext but also export auxiliary information generated from the randomness used in the former encryption process. Motivated by the above problems, we adopt two-trapdoor HE for the twin-server model, where $\mathcal{S}_{i\in[1,2]}$ holds a secret share $sk_{i\in[1,2]}$ of sk. To decrypt a ciphertext $[\![x]\!]$, two algorithms $\mathsf{PartDec}$ and $\mathsf{FullDec}$ are required successively, involving both secret shares sk_1, sk_2. $\mathsf{PartDec}$ is first called to obtain the partial decryption share $[x]_i$ with $sk_{i\in[1,2]}$ using Eq. 2. Then, $\mathsf{PartDec}$ is executed with $[x]_{i\in[1,2]}$ to obtain the plaintext x using Eq. 6. In this way, even if the adversary holds any one secret share $sk_{i\in[1,2]}$, sk's confidentiality cannot be compromised, which is proved in **Theorem** 1. The efficiency performance is shown in Section VII-B.

b) Model distribution: $\mathcal{S}_1$ releases the initial global model $[\![W^{(0)}]\!]$ to $\mathcal{U}$. Meanwhile, $\mathcal{S}_1$ executes $[W^{(0)}]_{sk_{s_i}} \leftarrow \mathsf{PartDec}_{sk_{s_i}}([\![W^{(0)}]\!])$ using the secret key share sk_{s_i}, and sends $[W^{(0)}]_{sk_{s_i}}$ to $\mathcal{U}$. Afterwards, $\mathcal{U}$ obtains $W^{(0)}$ using $\mathsf{PartDec}$ and $\mathsf{FullDec}$ algorithms for local training.

2) Local Training: In the t-th training iteration, each user $\mathcal{U}_i$ locally trains individual gradient $g_i^{(t)}$ using SGD, and sends encrypted local gradient $[\![g_i^{(t)}]\!]$ to $\mathcal{S}_1$. We show the detailed procedures in Algorithm 2, including benign training (for benign users $\mathcal{U}$) or local poisoning (for malicious users $\mathcal{U}^*$), gradient normalization and gradient encryption.

Algorithm 2 Local Training

Input: Encrypted global weight $[\![W^{(t)}]\!]$, local training data D_i ($i \in [1, n]$).
Output: Encrypted local gradients $[\![g_i^{(t)}]\!]$.

if $\mathcal{U}_i \notin \mathcal{U}^*$ **then**
  /*Benign training*/
  $\mathcal{U}_i$ trains $W^{(t)}$ on local data, and obtains local gradients g_i;
  /*Gradient normalization*/
  $\mathcal{U}_i$ normalizes individual gradients $g_i^{(t)}$ before sending them to $\mathcal{S}_1$;
else
  /*Model poisoning*/
  $\mathcal{U}_i$ launches model poisoning, and yields poisonous gradients $g_i^{*(t)}$;
return Encrypted local gradients $[\![g_i{}^{(t)}]\!]$ or $[\![g_i^{*(t)}]\!]$.

a) Benign training: Each gradient $g_i^{(t)}$ may comprise of thousands of elements. In order to reduce the communication overhead and emphasize the update direction of a local gradient, $g_i^{(t)}$ is collected with the selective SGD [5]. If the value of any element $x \in g_i^{(t)}$ is lower than the threshold κ (implying that $x^{(t)}$ is a potentially low-contribution element for the gradient update), it is thus not contained in $g_i^{(t)}$ [22].

Conversely, if $x^{(t)} - x^{(t-1)} \geq \kappa$, then the gradient element $x^{(t)}$ is collected in $g_i^{(t)}$. U_i uses "0" to pad out the m-dimensional gradient vector $g_i^{(t)} \leftarrow \mathbb{R}^m$.

Then, each user $\mathcal{U}_i$ normalizes local gradients $g_i^{(t)}$ within [0, 1] [30]. The normalized gradient is computed such that

$$x = \frac{x}{\|g_i^{(t)}\|}, \quad s.t., \; x \in g_i^{(t)}. \tag{11}$$

As the two-trapdoor HE is operated over integers, we adopt the approximation method Appr to convert a float-point number to an integer using the precision degree deg. For example, $\mathcal{U}_i$ obtains $g_i^{(t)} = [x_1, x_2, \ldots, x_8] = [1, 0, 1, 0, 0.3, 0, 0, 0.4]$. The normalized gradients are obtained as $g_i^{(t)} = [\frac{2}{3}, 0, \frac{2}{3}, 0, 0.2, 0, 0, \frac{4}{15}]$. Then, we obtain $g_i^{(t)} = [66667, 0, 66667, 0, 20000, 0, 0, 26667]$ for $deg = 10^5$ such that

$$x' = \mathsf{Appr}(x) = \lfloor x \cdot deg \rceil, \quad x \in g_i^{(t)}. \tag{12}$$

Finally, $\mathcal{U}_i$ sends $[\![g_i^{(t)}]\!]$ to $\mathcal{S}_1$, where $[\![g_i^{(t)}]\!] = \{[\![x'_1]\!], [\![x'_2]\!], \ldots, [\![x'_m]\!]\}$, and $[\![x']\!] \leftarrow \mathsf{Enc}_{pk}(x')$ $(x' \in g_i^{(t)})$.

b) Model poisoning attack: Under the adversarial setting, a Byzantine adversary $\mathcal{A}$ manipulates a set of malicious users $\mathcal{U}^* \in \mathcal{U}$ to launch a model poisoning attack. For a successful deployment of model poisoning, it requires as few as possible malicious users $\mathcal{U}_i^*$ to be involved. The collusion among $\mathcal{U}_j^*$ is to corrupt PPFL as shown in Eq. 10. To launch the model poisoning attack, $\mathcal{U}^*$ crafts local poisonous gradients g^* that deviate from the direction of benign gradients. Noticeably, to enhance the impact of poisonous gradients in FL, a local poisonous gradient g_j^* may not be normalized since a large magnitude of g_j^* can greatly affect the direction of the aggregated gradient. Finally, each malicious user $\mathcal{U}^*$ encrypts the local poisonous gradient using the public key pk and sends $[\![g_j^*]\!]$ to $\mathcal{S}_1$. Due to the indistinguishability of encrypted data on both $\mathcal{S}_1$ and $\mathcal{S}_2$, it provides the covertness for the encrypted model poisoning attack.

3) Privacy-Preserving Defense Strategy: At the beginning of training, all local gradients are assumed to be benign, *i.e.,* local training of all users is not poisoned. It is a stationary assumption that pristine local gradients $g_{i\in[1,n]}^{(1)}$ (*i.e.,* $t = 1$) are not corrupted by the adversary. The assumption is underpinned in [31]. The privacy-preserving defense strategy is elaborated in Algorithm 3, including normalization judgment, secure cosine similarity and Byzantine-tolerance aggregation. Importantly, it does not make assumptions about how poisonous gradients are generated and the true data distribution in FL, thus it provides guarantees under worst-case adversaries and heterogeneous data in FL. To resist encrypted model poisoning attacks, we propose the privacy-preserving defense strategy in ShieldFL, which is based on cosine similarity as stated in Proposition 1.

Proposition 1: Let $[\![g_1]\!], \ldots, [\![g_n]\!]$ be encrypted local gradients. The cosine similarity value $\cos_{i,j}$ $(i, j \in [1, n])$ of any two encrypted gradients $[\![g_i]\!]$ and $[\![g_j]\!]$ is defined such that

$$\cos_{i,j} = \frac{g_i \odot {g_j}^T}{\|g_i\| \cdot \|g_j\|} = g_i \odot {g_j}^T, \quad s.t., \|g_i\| = \|g_j\| = 1. \tag{13}$$

Algorithm 3 Privacy-Preserving Defense Strategy

Input: The local gradients $\{[\![g_1^{(t)}]\!], \ldots, [\![g_n^{(t)}]\!]\}$.
Output: Aggregated gradients $[\![g^{(t)}]\!]$.

if $t == 1$ **then**
  $[\![g^{(t)}]\!] \leftarrow [\![g_1^{(t)}]\!] \cdot [\![g_2^{(t)}]\!] \cdot \ldots \cdot [\![g_n^{(t)}]\!]$;
  $[\![g^{(t)}]\!] \leftarrow \mathsf{HE.Trun}([\![g^{(t)}]\!], n)$;
for $i \in [1, n]$ **do**
  */*Normalization judgment*/*
  $sum \leftarrow \mathsf{SecJudge}([\![g_i^{(t)}]\!])$;
  if $sum/deg^2 == 1$ **then**
    */*Secure cosine similarity*/*
    $\cos_i \leftarrow \mathsf{SecCos}([\![g_i^{(t)}]\!], [\![g^{(t-1)}]\!])$;
Find the gradient $[\![g^*]\!]$ with the lowest cosine similarity cos as the baseline of poisonous gradients;
*/*Byzantine-tolerance aggregation*/*
$[\![g^{(t)}]\!] \leftarrow [\![0]\!]$;
for $i \in [1, n]$ **do**
  $\cos_i \leftarrow \mathsf{SecCos}([\![g_i^{(t)}]\!], [\![g^*]\!])$;
  $\eta_i \leftarrow deg - \cos_i$;
for $i \in [1, n]$ **do**
  $\eta_i \leftarrow \lceil \frac{\eta_i}{\sum_{i=1}^n \eta_i} \cdot deg \rfloor$;
  $[\![g^{(t)}]\!] \leftarrow [\![g^{(t)}]\!] \cdot [\![g_i^{(t)}]\!]^{\eta_i}$ based on Eq. 5;
return The aggregated gradient $[\![g^{(t)}]\!]$.

"$\odot$" means the inner product, and "T" means the vector transposition [17]. If and only if two gradients g_i and g_j are normalized vectors, their cosine similarity satisfies $\cos_{i,j} = g_i \odot {g_j}^T$.

a) Normalization judgment: Upon receiving encrypted local gradients, $\mathcal{S}_1$ determines whether local gradients $[\![g_i^{(t)}]\!] = \{[\![x'_1]\!], [\![x'_2]\!], \ldots, [\![x'_m]\!]\}$ are normalized. The procedure $\mathsf{SecJudge}([\![g_i^{(t)}]\!])$ involves $\mathsf{HE.Mul}$ with $\mathcal{S}_2$. First, $\mathcal{S}_1$ and $\mathcal{S}_2$ implement $[\![x'^2_k]\!] \leftarrow \mathsf{HE.Mul}([\![x'_k]\!], [\![x'_k]\!])$. Then, $\mathcal{S}_1$ obtains $[\![sum]\!] = [\![x'^2_1 + \ldots + x'^2_m]\!]$ and decrypts sum in cooperation with $\mathcal{S}_2$. If $sum/deg^2 = 1$, then $[\![g_i^{(t)}]\!]$ is accepted by $\mathcal{S}_1$. Otherwise, $\mathcal{S}_1$ aborts $[\![g_i^{(t)}]\!]$ that has not been normalized. The specified procedure of $\mathsf{SecJudge}([\![g_i^{(t)}]\!])$ is described in Fig. 5. Note that sum is expanded as $sum = \sum_{j=1}^m x'^2_j = \sum_{j=1}^m (x_j \cdot deg)^2 = deg^2$, where $\sum_{j=1}^m x_j^2 = 1$.

b) Secure cosine similarity: In the t-th training iteration $(t > 1)$, $\mathcal{S}_1$ obtains the aggregated gradient $[\![g^{(t-1)}]\!]$ of the last iteration, which is used to distinguish encrypted poisonous gradients with the returned low cosine similarity values cos. To implement the cosine similarity among encrypted gradients, we design secure cosine similarity SecCos between $\mathcal{S}_1$ and $\mathcal{S}_2$. Given two encrypted local gradients $[\![g_a]\!]$ and $[\![g_b]\!]$, $\mathcal{S}_1$ and $\mathcal{S}_2$ construct $\mathsf{SecCos}([\![g_a]\!], [\![g_b]\!]) \rightarrow \cos_{ab}$, $\cos_{ab} \in [-deg, deg]$. The specified procedure is demonstrated in Fig. 6. Then, $\mathcal{S}_1$ implements $\mathsf{SecCos}([\![g_i^{(t)}]\!], [\![g^{(t-1)}]\!])$ to obtain the cosine similarity $\cos_i$ between each local gradient $[\![g_i^{(t)}]\!]$ $(i \in [1, n])$ and the aggregated gradient $[\![g^{(t-1)}]\!]$ of the $(t-1)$-th iteration.

Implementation of SecJudge($[\![g_i]\!]$)

For an encrypted local gradient $[\![g_i]\!] = \{[\![x'_1]\!], [\![x'_2]\!], ..., [\![x'_m]\!]\}$ submitted to $\mathcal{S}_1$ ($i \in [1, n]$):

- @$\mathcal{S}_1$: To protect data privacy, $\mathcal{S}_1$ first chooses m random noises $r_{k\in[1,m]} \leftarrow \mathbb{Z}_N^*$, and blinds r_k to $[\![x'_k]\!]$ using Eq. 5 such that
$$[\![\overline{x}_k]\!] = [\![x'_k]\!] \cdot [\![r_k]\!] = [\![x'_k + r_k]\!]. \tag{14}$$
Then, $\mathcal{S}_1$ implements the partial decryption by calling $[\overline{x}_k]_1 \leftarrow \mathsf{PartDec}_{sk_1}([\![\overline{x}_k]\!])$, and sends $\{[\![\overline{x}_1]\!], ..., [\![\overline{x}_m]\!]\}$ and $\{[\overline{x}_1]_1, ..., [\overline{x}_m]_1\}$ to $\mathcal{S}_2$ ($k \in [1, m]$).
- @$\mathcal{S}_2$: To decrypt the blinded number $\overline{x}_k$, $\mathcal{S}_2$ calls $[\overline{x}_k]_2 \leftarrow \mathsf{PartDec}_{sk_2}([\![\overline{x}_k]\!])$ and $\overline{x}_k \leftarrow \mathsf{FullDec}([\overline{x}_k]_1, [\overline{x}_k]_2)$. Then, $\mathcal{S}_2$ returns $[\![\Sigma\overline{x}_k{}^2]\!]$ to $\mathcal{S}_1$ ($k \in [1, m]$) such that $[\![\Sigma\overline{x}_k{}^2]\!] \leftarrow \mathsf{Enc}_{pk}(\overline{x}_1{}^2 + , ..., + \overline{x}_m{}^2)$.
- @$\mathcal{S}_1$: To remove the random noises $r_{k\in[1,m]}$, $\mathcal{S}_1$ implements
$$\begin{aligned}[\![sum]\!] &= [\![x'^2_1 + ... + x'^2_m]\!] \\ &= [\![\Sigma(x'_k + r_k)^2]\!] \cdot ([\![x'_1]\!]^{2r_1} \cdot, ..., \cdot [\![x'_m]\!]^{2r_m} \cdot [\![\Sigma r_k^2]\!])^{N-1} \\ &= [\![\Sigma\overline{x}_k{}^2]\!] \cdot [\![\Sigma(2r_k x_k + r_k^2))]\!]^{N-1},\end{aligned} \tag{15}$$
and sends $[\![sum]\!]$ to $\mathcal{S}_2$.
- @$\mathcal{S}_2$: $[sum]_2 \leftarrow \mathsf{PartDec}_{sk_2}([\![sum]\!])$ is executed to return $\mathcal{S}_1$.
- @$\mathcal{S}_1$: To obtain the final result sum, $\mathcal{S}_1$ uses PartDec and FullDec. Note that the precise degree of sum is expanded as deg^2 after HE.Mul. If $sum/deg^2 = 1$, it means the local gradient g_i is normalized. $\mathcal{S}_1$ accepts g_i for further training. Otherwise, g_i is aborted without gradients normalization as
$$g_i = \begin{cases} \text{Accept}, & \text{If } sum/deg^2 = 1, \\ \text{Abort}, & \text{Otherwise.} \end{cases}$$

Fig. 5. Detailed procedure of SecJudge.

c) Byzantine-tolerance aggregation: To avoid misidentifying poisonous gradients, ShieldFL designs a Byzantine-tolerance aggregation mechanism, which is demonstrated as follows.

- $\mathcal{S}_1$ selects the poisonous baseline $[\![g^*]\!]$ among local gradients $[\![g^{(t)}_{i\in[1,n]}]\!]$, which is identified to have the lowest cosine similarity with the aggregated gradient $[\![g^{(t-1)}]\!]$ of the last iteration.
- $\mathcal{S}_1$ and $\mathcal{S}_2$ then execute $\cos_j \leftarrow \mathsf{SecCos}([\![g^*]\!], [\![g^{(t)}_j]\!])$ to measure the similarity between the poisonous baseline $[\![g^*]\!]$ and other local gradients $[\![g^{(t)}_{j\in[1,n]}]\!]$. The more similar the two gradients are, the more likely it is a poisonous gradient. Note that $\cos_j \in [-deg, deg]$ as the precision degree deg is involved for HE computations.
- $\mathcal{S}_1$ sets the confidence $\eta_j \leftarrow 1 * deg - \cos_j$ instead of discarding abnormal gradients, We use the confidence η_j as a discriminator to estimate all local gradients. $\mathcal{S}_1$ implements the secure aggregation using η_j such that
$$\begin{aligned}\eta &\leftarrow \sum_{i=1}^{n} \eta_j, \ \eta_j \leftarrow \lfloor \frac{\eta_j}{\eta} \cdot deg \rceil, \\ [\![g^{(t)}]\!] &= [\![g^{(t)}_1]\!]^{\eta_1} \cdot, \ldots, \cdot [\![g^{(t)}_n]\!]^{\eta_n}, \\ [\![g^{(t)}]\!] &\leftarrow \mathsf{HE.Trun}([\![g^{(t)}]\!], deg),\end{aligned}$$
where $g^{(t)} = \sum_{i=1}^{n} \frac{\eta_i}{\eta} \cdot g_i^{(t)}$.

d) Weight update: $\mathcal{S}_1$ returns the aggregated gradient $[\![g^{(t)}]\!]$ and $[g^{(t)}]_{s_i}$ using PartDec with the secret key share sk_{s_i} to $\mathcal{U}_i$. Then, $\mathcal{U}_i$ obtains $g^{(t)}$ by using PartDec and FullDec with sk_{u_i}. To remove the noise deg involved in $g^{(t)}$, $\mathcal{U}_i$ executes $x' = \frac{x}{deg}$, $x \in g^{(t)}$. Finally, $\mathcal{U}_i$ executes the weight update to tune the local model $W_i^{(t)}$ according to Eq. 9. It is guaranteed that **Algorithm** 3 converges and terminates within a finite number $\top$ of iterations, as stated in **Theorem** 3.

Implementation of SecCos($[\![g_a]\!], [\![g_b]\!]$)

Given two encrypted gradients $[\![g_a]\!] = \{[\![x'_{a_1}]\!], [\![x'_{a_2}]\!], ..., [\![x'_{a_m}]\!]\}$ and $[\![g_b]\!] = \{[\![x'_{b_1}]\!], [\![x'_{b_2}]\!], ..., [\![x'_{b_m}]\!]\}$:

- @$\mathcal{S}_1$: Once receiving $[\![g_a]\!]$ and $[\![g_b]\!]$, $\mathcal{S}_1$ first selects m random noises $r_{k\in[1,m]} \leftarrow \mathbb{Z}_N^*$, and blinds r_k to $[\![g_a]\!]$ and $[\![g_b]\!]$. As computed in Eq. 14, $[\![\overline{g}_a]\!] = \{[\![\overline{x}_{a_1}]\!], ..., [\![\overline{x}_{a_m}]\!]\}$ and $[\![\overline{g}_b]\!] = \{[\![\overline{x}_{b_1}]\!], ..., [\![\overline{x}_{b_m}]\!]\}$ are obtained. Then, $\mathcal{S}_1$ executes
$$[\overline{x}_{a_k}]_1 \leftarrow \mathsf{PartDec}_{sk_1}([\![\overline{x}_{a_k}]\!]), \ [\overline{x}_{b_k}]_1 \leftarrow \mathsf{PartDec}_{sk_1}([\![\overline{x}_{b_k}]\!]), \quad s.t., \ [\![\overline{x}_{a_k}]\!] \in [\![\overline{g}_a]\!], [\![\overline{x}_{b_k}]\!] \in [\![\overline{g}_b]\!],$$
and sends $[\overline{g}_a]_1, [\overline{g}_b]_1$, $[\![\overline{g}_a]\!], [\![\overline{g}_b]\!]$ to $\mathcal{S}_2$.
- @$\mathcal{S}_2$: Once obtaining these encrypted numbers, $\mathcal{S}_2$ calls PartDec and FullDec to obtain $\overline{g_a}$ and $\overline{g_b}$. Then, $\mathcal{S}_2$ implements
$$\overline{\cos}_{ab} = \overline{g_a} \odot \overline{g_b} = \Sigma_{k=1}^{m} \overline{x}_{a_k} \cdot \overline{x}_{b_k}. \tag{16}$$
$\mathcal{S}_2$ calls the Enc algorithm to return $[\![\overline{\cos}_{ab}]\!]$ to $\mathcal{S}_1$.
- @$\mathcal{S}_1$: To remove the noise r_k in $[\![\overline{\cos}_{ab}]\!]$, $\mathcal{S}_1$ implements Eq. 15 to obtain the final cosine similarity $[\![\cos_{ab}]\!]$.
- @$\mathcal{S}_2$: The decryption share $[\cos_{ab}]_2 \leftarrow \mathsf{PartDec}_{sk_2}([\![\cos_{ab}]\!])$ is executed to return $\mathcal{S}_1$.
- @$\mathcal{S}_1$: To obtain $\cos_{ab}$ between $[\![g_a]\!]$ and $[\![g_b]\!]$ as shown in Eq. 13, $\mathcal{S}_1$ computes the cosine similarity $\cos_{ab}$ with $[\cos_{ab}]_2 \leftarrow \mathsf{PartDec}_{sk_2}([\![\cos_{ab}]\!])$ and $\cos_{ab} \leftarrow \mathsf{FullDec}([\cos_{ab}]_1, [\cos_{ab}]_2)$. Note that the precise degree of $\cos_{ab}$ is expanded as deg^2 after HE.Mul. To keep the degree deg consistent, the final result is truncated as $\cos_{ab} \leftarrow \lfloor \frac{\cos_{ab}}{deg} \rceil$.

Fig. 6. Detailed procedure of SecCos.

Remark: Here, we consider that the privacy of the cosine similarity $\cos_j$ needn't be protected. The reason is that $\cos_j$ measures the cosine similarity between local gradients $[\![g^{(t)}_{j\in[1,n]}]\!]$ and the aggregated gradient $[\![g^{(t-1)}]\!]$. It cannot reveal the distribution of gradients $[\![g^{(t)}_{j\in[1,n]}]\!]$ and $[\![g^{(t-1)}]\!]$ even if $\cos_j$ is public. Besides, it cannot capture information of the norm magnitude and direction of local gradients $[\![g^{(t)}_{j\in[1,n]}]\!]$ without knowing the original information from encrypted gradients $[\![g^{(t)}_{j\in[1,n]}]\!]$ and $[\![g^{(t-1)}]\!]$. Consequently, existing data reconstruction attacks [32], [33] are unable to recover data points with limited information of $\cos_j$.

VI. Theoretical Analysis

In this section, we prove the security of ShieldFL, and analyze the computational and communication complexity in ShieldFL.

A. Security Analysis

In the *honest-but-curious* setting, we prove that ShieldFL achieves IND-CPA security [34] with the security definition of ShieldFL and following lemmas. The notion of ShieldFL

security is also captured using the real-world versus ideal-world game. In this section, we provide a hybrid argument to demonstrate the execution of ShieldFL, which does not leak any sensitive information about local gradients.

Definition 4 (ShieldFL Security): Consider the following game between an adversary $\mathcal{A}$ and a Probabilistic Polynomial-Time (PPT) simulator $\mathcal{A}^*$, let a protocol $\prod$ be secure if any view REAL of $\mathcal{A}$ in the real world is computationally indistinguishable from the view IDEAL of $\mathcal{A}^*$ in the ideal world. For all inputs of local gradients $[\![g_i^{(t)}]\!]$ and intermediate results sum, cos, it holds

$$\mathsf{REAL}_{\mathcal{A}}^{\prod}(g_i^{(t)}, sum, \cos) \stackrel{c}{\equiv} \mathsf{IDEAL}_{\mathcal{A}^*}^{\prod}(g_i^{(t)}, sum, \cos).$$

Note that "$\stackrel{c}{\equiv}$" represents computational indistinguishability.

Lemma 1: If a randomness r is uniformly distributed on $\mathbb{Z}_N$ and also independent from any plaintext $x \in \mathbb{Z}_N$, then $x \pm r \mod N$ is also uniformly random and independent from x [35].

Lemma 2: Secure multiplication protocol $\mathsf{HE.Mul}$ is secure under the *honest-but-curious* setting [27].

Lemma 3: If all subprotocols are simulatable, then the protocol can be perfectly simulatable [36].

The detailed proofs of the above lemmas are shown in [27], [35], [36]. The function privacy and semantic privacy of the two-trapdoor HE system against chosen-plaintext attacks provide the strong security in ShieldFL. As defined in Definition 4, we just need to prove the confidentiality of the privacy-preserving defense strategy. Due to the fact that other protocols in ShieldFL are only executed locally, it is trivial to prove its confidentiality. Our proof of data confidentiality is built on Lemma 1, Lemma 2 and Lemma 3. In the threat model, the cloud servers $\mathcal{S}_1$ and $\mathcal{S}_2$ are non-colluding. The collusion between users and servers is inexistent since the collusion can help $\mathcal{S}_1$ and $\mathcal{S}_2$ to easily identify local model poisoning from malicious users, and leak local information from benign users. Besides, malicious users $\mathcal{U}^*$ are colluded to evade the defense strategy and launch the model poisoning attack in FL, but they still have no knowledge about local gradients from benign users except the aggregated gradient. As demonstrated above, we present the following theorems.

Theorem 1 (Secret Key Confidentiality): Given any pair $(sk_1, sk_2) \leftarrow \mathsf{KeySplit}(sk)$, secret key shares $sk_{k\in[1,2]} \in \mathbb{Z}_N$ are computationally indistinguishable.

Proof: The view of $\mathsf{REAL}_{\mathcal{A}}^{\prod}$ consists of all internal states and messages during the execution of the protocol $\prod$. Given the security parameters, a PPT simulator $\mathcal{A}^*$ is defined with a series of (polynomially many) subsequent modifications to the random variable $\mathsf{REAL}_{\mathcal{A}}^{\prod}$. Thus, any two subsequent random variables are computationally indistinguishable. The detailed proof is demonstrated as follows.

In the view of $\mathsf{REAL}_{\mathcal{A}}^{\prod}$, we randomly split the secret key $sk = \lambda$ into two shares sk_1 and sk_2 using the $\mathsf{KeySplit}$ algorithm, which satisfies

$$sk_1 + sk_2 \equiv 0 \mod \lambda, sk_1 + sk_2 \equiv 1 \mod N.$$

Let $s = sk_1 + sk_2$. According to Chinese remainder theorem and $gcd(\lambda, N^2) = 1$, $\exists s \in \mathbb{Z}_N$ satisfies $s \equiv 0 \mod \lambda$, $s \equiv 1 \mod N$, $s \equiv 0 \mod \lambda, s \equiv 1 \mod N$.

We only require to choose two random integers sk_1 and sk_2 satisfying $sk_1 + sk_2 = s$. If a random number $r \leftarrow \mathbb{Z}_{\lambda N}$ is chosen as sk_1, then $sk_2 = s - r \mod \lambda N$. Based on Lemma 1, both r and $s - r \mod N$ are the uniform distribution and independent from s. Even $\mathcal{A}$ receives any one secret key share sk_k ($k \in [1, 2]$), the other share and s cannot be inferred. For any $r \in \mathbb{Z}_N$, sk_1 and sk_2 are computationally indistinguishable.

In the view of $\mathsf{IDEAL}_{\mathcal{A}^*}^{\prod}$, we simulate a uniformly random $sk'_{k\in[1,2]}$ chosen by $\mathcal{A}^*$ instead of using original secret key shares $sk_{k\in[1,2]}$ to implement decryption algorithms $\mathsf{PartDec}$ and $\mathsf{FullDec}$. As $sk'_{k\in[1,2]}$ is identical to the distribution of an equivalent number of any given secret key shares $sk_{k\in[1,2]}$, making the views of $\mathcal{A}$ in ideal and real executions are indistinguishable. □

Theorem 2 (Data Confidentiality): ShieldFL securely implements the privacy-preserving defense strategy between $\mathcal{S}_1$ and $\mathcal{S}_2$ over encrypted gradients in the presence of $\mathcal{A}$.

Proof: ShieldFL raises privacy concerns since all trained gradients are accessible to each user $\mathcal{U}_i$, especially for malicious users $\mathcal{U}^*$. As the data distribution can be inferred from gradients, it requires original gradients and secure operations for data confidentiality. With respect to the above lemmas, we prove that ShieldFL is computationally distinguishable from the ideal-world view of $\mathcal{A}^*$ and the real-world view of $\mathcal{A}$. The standard hybrid argument is used to prove the theorem [2]. For any honest-but-curious server $\mathcal{S}_1$ and $\mathcal{S}_2$, $\mathsf{REAL}_{\mathcal{A}}^{\prod}$ contains intermediate parameters and encrypted local gradients $[\![g_i]\!]$ received from other entities during the execution of protocols. To protect the sensitive information about encrypted local gradients and intermediate parameters, it is important to ensure the privacy of secret key sk. If $\mathcal{A}$ obtains a secret key share sk_i from a user or a server, the probability of inferring sk is equivalent to the negligible probability in **Theorem 1**. There is no way that any secret key share can leak the original information of secret key sk. Besides, we construct a PPT simulator $\mathcal{A}^*$ to analyze the each process in the privacy-preserving defense strategy that comprises of *normalization judgement*, *secure cosine similarity* and *Byzantine-tolerance aggregation*. The detailed proof is demonstrated below.

Hyb_0 We initialize a series of random variables that are indistinguishable from $\mathsf{REAL}_{\mathcal{A}}^{\prod}$ during the real execution process of the protocol.

Hyb_1 In this hybrid, we change the behavior of simulated users $\mathcal{U}_i \in \mathcal{U}$, where $\mathcal{U}_i$ encrypts a selected random vector of random variables χ_g using a uniformly random public key pk chosen by the simulator $\mathcal{A}^*$ instead of the original local gradient $g_i^{(t)}$. Under the decisional composite residuosity assumption, two non-colluding $\mathcal{S}_1$ and $\mathcal{S}_2$ cannot distinguish the view of χ_g from the view of $g_i^{(t)}$ with the IND-CPA security property of two-trapdoor HE in this hybrid.

Hyb_2 In this hybrid, we substitute all ciphertexts encrypted by $\mathcal{U}$. As only the original contents of ciphertexts have changed, the IND-CPA security of the two-trapdoor HE guarantees that this hybrid is indistinguishable from the previous one.

Hyb_3 In this hybrid, we change the input of the protocols SecJudge and SecCos executed by two non-colluding servers $\mathcal{S}_1$ and $\mathcal{S}_2$ with random variables $[\![\chi]\!]$ instead of blinded gradients $[\![x]\!] \cdot [\![r]\!]$, where $[\![x]\!] \in [\![g_i^{(t)}]\!]\cdot$. The IND-CPA security property of the two-trapdoor HE and the non-colluding setting guarantees that this hybrid is indistinguishable from the previous one.

Hyb_4 In this hybrid, local encrypted gradients $[\![g_i^{(t)}]\!]$ are blinded with random values, which can be perfectly simulated by $\mathcal{A}^*$ with random numbers $r \leftarrow \mathbb{Z}_N^*$. Since the intermediate parameters are blinded with uniformly random numbers, the perturbed intermediate parameters also follow the uniform distribution. The properties of additive homomorphic thus guarantee that the distribution of any intermediate variables is identical to the distribution of an equivalent number of intermediate variables. Thus, the output of $\mathcal{A}^*$ cannot be distinguished by $\mathcal{A}$ in polynomial time, making this hybrid identically distributed to the previous one.

Hyb_5 In this hybrid, the aggregated gradient $[\![g^{(t)}]\!]$ computed using Eq. 5, which is based on additive homomorphic. $\mathcal{A}^*$ holds the view $\mathsf{Ideal}_{\mathcal{A}^*}^{\prod} = \{[\![g_i^{(t)}]\!], sum, \cos, [\![g^{(t)}]\!]\}$, where plaintexts *sum* and cos are the inner product of encrypted local gradients after SecJudge and SecCos. As the inputs $[\![g_i^{(t)}]\!]$ are encrypted and intermediate results are protected by noises, *sum* and cos cannot leak users' data privacy without knowing any information of inputs and intermediate computations. Among the elements of intermediate computation, the blinded number $\overline{x} \in g_i^{(t)}$ are still uniformly random, which is consistent with the previous hybrid. Hence, this hybrid is indistinguishable from the previous one. One can deduce that $\mathsf{REAL}_{\mathcal{A}}^{\prod}(g_i^{(t)}, sum, \cos) \stackrel{c}{\equiv} \mathsf{IDEAL}_{\mathcal{A}^*}^{\prod}(g_i^{(t)}, sum, \cos)$, which proves the real views of the interactive protocols and simulated views by $\mathcal{A}^*$ are simulatable and computationally distinguishable.

The argument above proves that the output of $\mathsf{IDEAL}_{\mathcal{A}^*}^{\prod}$ is computationally indistinguishable from the output of $\mathsf{REAL}_{\mathcal{A}}^{\prod}$. Therefore, it indicates that ShieldFL guarantees data confidentiality. □

B. Convergence Analysis

We provide provable guarantees on the convergence of **Algorithm** 3. We first present the following assumptions, which are partly based on [37]. Note that the following theorem is stated without any assumptions on the training data distribution.

Assumption 1: (Strong Convexity and Smoothness): The local objective $\mathcal{L}_{i\in[1,n]}$ is all μ-strongly convex and L-smooth with $\mu > 0$, for any W and $\overline{W}$,

$$\mathcal{L}_i(W) \le \mathcal{L}_i(\overline{W}) + (W - \overline{W})^T \nabla \mathcal{L}_i(W) + \frac{L}{2}\|W - \overline{W}\|_2^2,$$
$$\mathcal{L}_i(W) \ge \mathcal{L}_i(\overline{W}) + (W - \overline{W})^T \nabla \mathcal{L}_i(W) + \frac{\mu}{2}\|W - \overline{W}\|_2^2$$

Assumption 2 (Bounded Gradients): Let $D_i^{(t)}$ be randomly and uniformly sampled from $\mathcal{U}_i$'s local data. The local gradients g_i is bounded by a constant c_i, i.e., *for all i,*

$$\mathbb{E}\|\nabla \mathcal{L}_i(W_i^{(t)}, D_i^{(t)}) - \nabla \mathcal{L}_i(W_i^{(t)})\|^2 \le c_i.$$

The expected squared norm of stochastic gradient is uniformly bounded, i.e., *there exists a constant c_2 for all i and t such that*

$$\mathbb{E}\|\nabla \mathcal{L}_i(W_i^{(t)}, D_i^{(t)})\|^2 \le c_2.$$

Assumption 3: (Bound Between Benign and Poisonous Gradients): Let $\top$ be the total number of training iterations in ShieldFL. For an iteration counter K, the confidence η_{mal} of poisonous gradients drops by at least δ_{mal}, and the confidence η_{ben} of benign gradients increases by at least δ_{ben} such that

$$|\eta_{mal}^{(t)} - \eta_{mal}^{(t-K)}| \ge \delta_{mal}, \quad |\eta_{ben}^{(t)} - \eta_{ben}^{(t-K)}| \ge \delta_{ben},$$
$$s.t.\ t \in \{1, 2, \ldots, \top\}.$$

Assumption 1 is standard for the ℓ_2-norm regularized linear regression, logistic regression, and softmax classifier, which are used throughout the rest of the paper. **Assumption** 2 states that the local gradients $g_{i\in[1,n]}$ are bounded to the global gradients for all benign and poisonous users. **Assumption** 3 claims that ShieldFL has better ability to identify the poisonous gradients after K iterations.

Definition 5 (Non-IIDness): Let W_i^* and W^* be the minimum value of the local model and the global model, respectively. We define the term $\Gamma = W^* - \sum \eta_i W_i^*$ to quantify the degree of non-IIDness ($\Gamma \ge 0$). Γ obviously goes to 0 with IID data. Γ is nonzero with non-IID data. The higher the magnitude of Γ, the greater the degree of non-IIDness [37].

Theorem 3: Let Assumption 1 and 2 hold; and L, μ, c_i, c_2 be defined therein. Let $\gamma = \max(8\frac{L}{\mu}, 1)$, and u^* be the number of poisonous users $\mathcal{U}^*$. Then, ShieldFL satisfies that

$$\mathbb{E}[W^\top] - W^* \le \frac{2}{\mu^2} \cdot \frac{L}{\gamma + \top}\Big(\sum_{i=1}^{n} \eta_i^2 c_i^2 + 6L\Gamma + 8c_2^2 \sum_{k=1}^{u^*} \eta_k^{(0)} + \frac{\mu^2}{4}\|W^0 - W^*\|^2\Big), \quad (17)$$

Proof: Given a batch of local training data $D_i^{(t)} = \{x_{i_1}, x_{i_2}, \ldots, x_{i_b}\}$. The local objective of $\mathcal{U}_i$ is given as $\mathcal{L}_i(W_i^{(t)}, D_i^{(t)})$, where b is the batch size. The local gradient $g_i^{(t)} = \nabla \mathcal{L}_i(W_i^{(t)}, D_i^{(t)})$. The global gradient $g^{(t)}$ is defined as $g^{(t)} = \sum_{i=1}^n \eta_i g_i^{(t)}$ The global model $W^{(t+1)}$ is updated as $W_i^{(t+1)} \leftarrow W_i^{(t)} - lr^{(t)} g^{(t)}$, where η_i is the confidence of $\mathcal{U}_i$, $lr^{(t)} = \frac{1}{\mu(\gamma + t)}$ is the learning rate.

Based on above assumptions, we give the following lemmas, which are proved in [37].

Lemma 4: Let Assumption 1 and 2 hold. To bound the expectation of the global objective $\overline{W}^{(t+1)}$ in the $\top$-th iteration, we have

$$\mathbb{E}\|\overline{W}^{(t+1)} - W^*\|^2 \le (1 - \mu \cdot lr^{(t)})\mathbb{E}\|\overline{W}^{(t)} - W^*\|^2 + 6L\Gamma {lr^{(t)}}^2 + {lr^{(t)}}^2 \mathbb{E}\|g^{(t)} - \overline{g}^{(t)}\|^2 + 2\mathbb{E}\sum_{i=1}^{n} \eta_i \|\overline{W}^{(t)} - W_i^{(t)}\|^2.$$

TABLE III
COMPUTATION AND COMMUNICATION COMPLEXITY IN SHIELDFL

Phase	ShieldFL	
	Compu.	Comm.
Local training ($@\mathcal{U}$)	$\mathcal{O}(m\mathsf{T}_{\mathsf{Enc}})$	$\mathcal{O}(m\|X\|)$
Normalization judgement	$\mathcal{O}(nm(\mathsf{T}_{\mathsf{Mul}}+\mathsf{T}_{\mathsf{PDec}}+\mathsf{T}_{\mathsf{Add}}))$	$\mathcal{O}(4nm\|X\|)$
Secure cosine similarity	$\mathcal{O}(nm(\mathsf{T}_{\mathsf{Mul}}+\mathsf{T}_{\mathsf{PDec}}+\mathsf{T}_{\mathsf{Add}}))$	$\mathcal{O}(4nm\|X\|)$
Byzantine tolerance	$\mathcal{O}(nm(\mathsf{T}_{\mathsf{Mul}}+\mathsf{T}_{\mathsf{PDec}})+n\mathsf{T}_{\mathsf{Add}})$	$\mathcal{O}(4nm\|X\|)$

The abbreviation for communication complexity is "Comm.". The abbreviation for computation complexity is "Compu.".

Lemma 5: Let Assumption 2 hold; it follows that $\mathbb{E}\|g^{(t)}-\overline{g}^{(t)}\|^2 \leq \sum_{i=1}^{n}\eta_i^2 c_i^2$, *where* $g^{(t)}=\sum_{i=1}^{n}\eta_i g_i^{(t)}$, *and* $\overline{g}^{(t)}$ *is the gradient obtained by a batch of the whole training data* $D_1\cup D_2\cup\ldots\cup D_n$.

Lemma 6: Let Assumption 3 hold. Assume within E communication steps, there exists $t_0 < t$ *satisfying* $t-t_0 \leq E-1$ *and* $W_i^{(t_0)}=\overline{W}^{(t_0)}$. *Notice that* $lr^{(t)}$ *is non-increasing and* $lr^{(t)} \leq 2lr^{(t)}$, *we have* $\mathbb{E}\sum_{i=1}^{n}\eta_i\|\overline{W}^{(t)}-W_i^{(t)}\|^2 \leq 4\eta_i^2(E-1)^2g_2^2+4\sum_{k=1}^{u^*}\eta_k^{(0)}(E-1)^2lr^{(t)^2}$, *where* $\eta_k^{(0)}$ *is the initial confidence of poisonous users* $\mathcal{U}^*$.

By the L-smoothness, it follows that

$$\mathbb{E}[W^{\top}]-W^* \leq \frac{L}{2}\mathbb{E}\|\overline{W}^{(t)}-W^*\|^2 \tag{18}$$

From Eq. 18 and Lemma 4 to 6, we can derive that $\mathbb{E}[W^{\top}]-W^* \leq \frac{2}{\mu^2}\cdot\frac{L}{\gamma+\top}(\sum_{i=1}^{n}\eta_i^2c_i^2+6L\Gamma+8c_2^2\sum_{k=1}^{u^*}\eta_k^{(0)}+\frac{\mu^2}{4}\|W^0-W^*\|^2)$.

The derived proofs of key lemmas are shown in [37]. The above demonstration proves that a Byzantine-robust aggregation is guaranteed to converge in FL. The convergence speed is $\mathcal{O}(\frac{1}{\top})$. The increase of the number u^* of poisonous users $\mathcal{U}^*$ leads to an increases of the bound. □

C. Complexity Analysis

To evaluate the efficiency of ShieldFL, we discuss the complexity in the privacy-preserving defense strategy for each training iteration, as shown in TABLE III. Let n be the number of users, and m be the number of dimensions in local gradients. Let $|X|$ be the communication complexity of an encrypted number. $\mathsf{T}_{\mathsf{Mul}}$, $\mathsf{T}_{\mathsf{PDec}}$, and $\mathsf{T}_{\mathsf{Add}}$ are the time complexities of HE.Mul, PartDec and secure addition, where $\mathsf{T}_{\mathsf{Mul}} > \mathsf{T}_{\mathsf{PDec}} > \mathsf{T}_{\mathsf{Add}}$.

The local training is operated offline, during which each user $\mathcal{U}_i$ encrypts local gradients $[\![g_i^{(t)}]\!]$. The computational complexity and communication complexity depend on the dimension of gradients m. It costs $\mathcal{O}(m\cdot\mathsf{T}_{\mathsf{Enc}})$ for computation, and $\mathcal{O}(m|X|)$ for communication. The privacy-preserving defense strategy is executed online, which involves collaborative computations and communications between $\mathcal{S}_1$ and $\mathcal{S}_2$. In the normalization judgement process, the complexity mainly depends on the number of users n and the dimension of local gradients m. The computational complexity is $\mathcal{O}(n\cdot m(\mathsf{T}_{\mathsf{Mul}}+\mathsf{T}_{\mathsf{PDec}}+\mathsf{T}_{\mathsf{Add}}))$, and the communication complexity is $\mathcal{O}(4n\cdot m|X|)$. In the secure cosine similarity process, it costs $\mathcal{O}(n\cdot m(\mathsf{T}_{\mathsf{Mul}}+\mathsf{T}_{\mathsf{PDec}}+\mathsf{T}_{\mathsf{Add}}))$ for computational complexity, and costs $\mathcal{O}(4n\cdot m|X|)$ for communication complexity. In the Byzantine-tolerance mechanism, the computational complexity costs $\mathcal{O}(nm(\mathsf{T}_{\mathsf{Mul}}+\mathsf{T}_{\mathsf{PDec}}+\mathsf{T}_{\mathsf{Add}}))$, and the communication complexity costs $\mathcal{O}(4n\cdot m|X|)$, which also relies on the user size n and dimension of local gradients m.

VII. EXPERIMENTAL ANALYSIS

In this section, we analyze the experimental performance of ShieldFL from two perspectives: *i)* Evaluating the ShieldFL performance across heterogeneous data settings including non-IID and IID data. *ii)* Evaluating the ShieldFL performance against different model poisoning attacks (*i.e.,* targeted attacks and untargeted attacks). To highlight the advantages of ShieldFL, we construct the poisonous PPFL without defense as the baseline. For clarity, we simplify "the poisonous PPFL without defense" to "the baseline" below.

A. Testbed and Methodology

We evaluate our experiments with PyTorch running on a six-core Ubuntu 18.04 machine with Intel i7-9750H processor 4.50GHz and 16GB RAM. We adopt the two-trapdoor HE protocol with the 80-bit security level ($N = 1024$ bits) [38], which is implemented with Python 3.7. Besides, we simulate a number of users with lightweight threads, each of which implements a real-world PyTorch regression classifier for implementing PPFL in ShieldFL. The execution time given below is the average time of 10 trials.

1) Datasets, Data Settings and Model Architectures: We now introduce the datasets, data settings, and model architectures used in our experiments.

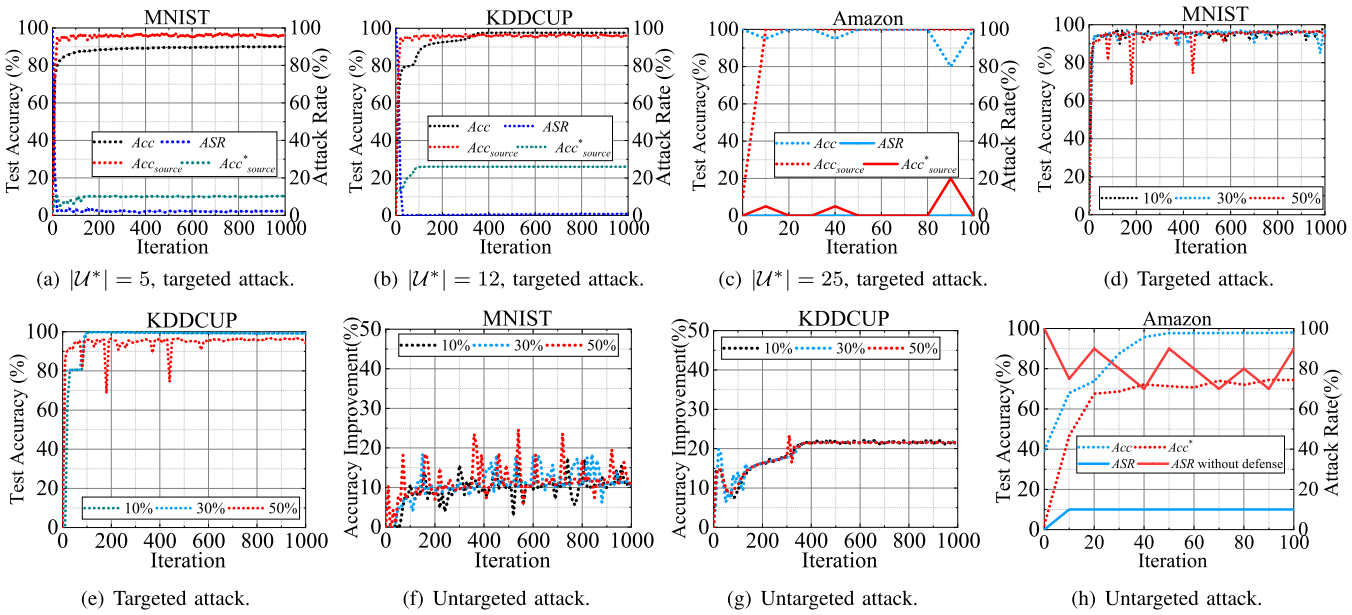
a) Datasets: We evaluate the performance of ShieldFL over three benchmark datasets. Each dataset is divided into a training set and a testing set with the proportion of 0.8 to 0.2.

- MNIST [39]: The dataset contains handwritten digits with 10 classes, provides 60, 000 training samples. Each sample is a $28*28$ pixel grayscale image.
- KDDCup99 [40]: The dataset includes network data with the size of 494, 020 that is divided into 23 classes with 41 features for network intrusion.
- Amazon [41]: The dataset includes a corpus of text from product reviews, which contains 1, 500 training samples divided into 50 classes with 10, 000 features.

b) Data settings: To evaluate ShieldFL, we build both IID settings and non-IID settings, respectively [42].

- Non-IID setting. The training data are divided by class, where each user stores training data samples belonging to a single class, *i.e.,* 1-class non-IID.
- IID setting. The training data are uniformly partitioned among $\mathcal{U}$, each user stores IID data.

c) Model architectures: We consider the widely used logistic regression classifier as the global model over different datasets, the mini-batch size is 50. In ShieldFL, there are 1, 000 training iterations for training on MNIST and KDDCup99, and 100 training iterations for training on Amazon. Besides, we set $deg = 10^5$ for the approximation function Appr.



(a) $|\mathcal{U}^*| = 5$, targeted attack. (b) $|\mathcal{U}^*| = 12$, targeted attack. (c) $|\mathcal{U}^*| = 25$, targeted attack. (d) Targeted attack.

(e) Targeted attack. (f) Untargeted attack. (g) Untargeted attack. (h) Untargeted attack.

Fig. 7. ShieldFL preformance with non-IID data.

2) Model Poisoning Attacks in PPFL: To evaluate ShieldFL against model poisoning, we construct the following two standard model poisoning attacks. Particularly, our adversarial attacks are set to the highest level. The entire data samples stored in a malicious user $\mathcal{U}^*$ are all poisonous, all of which are used to train encrypted local poisonous gradients $[\![g^*]\!]$.

- *Targeted attacks*: The targeted model poisoning attacks are launched to mislead specified data from a source class "i" to the specific target class "j", ensuring no false positives for other classes [8]. To simulate targeted attacks, the malicious users in $\mathcal{U}^*$ re-label the training data samples of the source class "i" as the target class "j", which are used to train local poisonous gradients.
- *Untargeted attacks*: The untargeted model poisoning attacks aim at increasing the false positives of a federated model to reduce its overall accuracy of predicting all data samples. To simulate untargeted attacks, local training samples held by $\mathcal{U}^*$ are re-labeled randomly as the incorrect classes, then the re-labeled samples are used to produce local poisonous gradients.

To evaluate the performance of ShieldFL, we simulate different levels of model poisoning attack, where local gradients are encrypted in PPFL. We deploy the different attack ratio Att_{ratio}. The Byzantine adversary corrupts the different ratio of malicious users $\mathcal{U}^* \in \mathcal{U}$. Let $Att_{ratio} = \frac{|\mathcal{U}^*|}{|\mathcal{U}|}$ be the attack ratio, $|\mathcal{U}|$ be the number of all users, and $|\mathcal{U}^*|$ be the number of malicious users that are corrupted by a Byzantine adversary. We simulate different attack scenarios with few malicious users $\mathcal{U}^*$ (*i.e.,* $Att_{ratio} = 10\%$) to the theoretic upper bound of all defense schemes (*i.e.,* $Att_{ratio} = 50\%$). TABLE IV depicts the adversarial settings, with which we evaluate ShieldFL on various datasets.

- For the MNIST dataset, 10 users are involved in ShieldFL, in which the number of malicious users $|\mathcal{U}^*|$ varies from 1 to 3 and 5. The targeted model poisoning attack aims to mislead data belonging to a source class "1" to the target class "7".
- For the KDDCup dataset, we set $|\mathcal{U}| = 23$, while the number of malicious users $|\mathcal{U}^*| \in [2, 7, 12]$. The targeted attack is launched to mislead data belonging to a source class "9" to the target class "11".
- For the Amazon dataset, we deploy $|\mathcal{U}| = 50$, and $|\mathcal{U}^*| = 25$. In our targeted attack, the source class is "1" and the target class is "7".

TABLE IV
ADVERSARIAL PARAMETERS

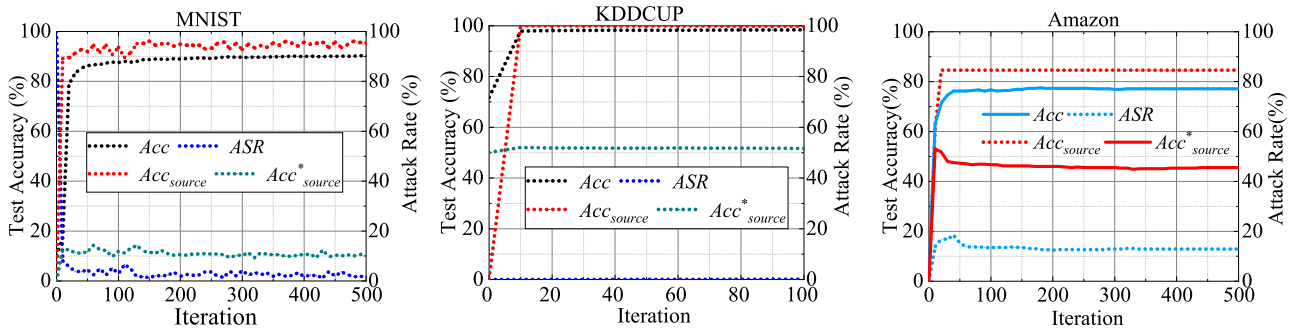
Datasets	$\lvert\mathcal{U}\rvert$	Att_{ratio}	Targeted attack	
			Source class	Target class
MNIST	10	$[10\%, 30\%, 50\%]$	"1"	"7"
KDDCup	23	$[10\%, 30\%, 50\%]$	"9"	"11"
Amazon	50	50%	"1"	"7"

Notes. To construct 1-class non-IID settings, we set $|\mathcal{U}|$ according to the class number in each dataset.

3) Performance Metrics: We use different metrics to measure the performance of ShieldFL. Acc is the overall accuracy. Acc_{source} is the accuracy of data with a specified source class. ASR is the attack success rate of ShieldFL. AI means the accuracy improvement in ShieldFL. We give specified definitions as follows:

Definition 6 (Attack Success Rate): The model poisoning attack is successful if the federated model incorrectly predicts testing samples in D_{test}. ASR is computed as $ASR = \frac{|D^*|}{|D_{test}|}$, where $|D^*|$ is the size of data with incorrect predictions, $|D_{test}|$ is the size of test data.

Definition 7 (Accuracy Improvement): Under model poisoning attacks, AI is the improvement of the overall accuracy achieved by ShieldFL over the baseline, which is defined as



(a) $Att_{ratio} = 50\%$, targeted attack. (b) $Att_{ratio} = 50\%$, targeted attack. (c) $Att_{ratio} = 50\%$, targeted attack.

(d) $Att_{ratio} = 50\%$, untargeted attack. (e) $Att_{ratio} = 50\%$, untargeted attack. (f) $Att_{ratio} = 50\%$, untargeted attack.

Fig. 8. ShieldFL performance with IID data.

TABLE V
PERFORMANCE COMPARISON IN THE NON-IID SETTING ($Att_{ratio} = 50\%$)

Datasets	Attack	Baseline		ShieldFL			
		Acc^*_{source}	Acc^*	Acc_{source}	Acc	AI_{source}	AI
MNIST	Targeted	10.3%	—	95.8%	90.1%	85.5%	—
	Untargeted	—	78.5%	—	89.4%	—	10.9%
KDDCUP	Targeted	25.8%	—	96.0%	97.6%	70.2%	—
	Untargeted	—	76.2%	—	90.7%	—	14.5%
Amazon	Targeted	0.2%	—	95.3%	99.7%	70.2%	—
	Untargeted	—	74.4%	—	97.8%	—	23.4%

$AI = Acc - Acc^*$, where Acc is the test accuracy of ShieldFL, and Acc^* is the test accuracy of the baseline. Let Acc_{source} be the source accuracy of ShieldFL, and Acc^*_{source} be the source accuracy of the baseline. AI_{source} is the improvement of source accuracy defined as $AI_{source} = Acc_{source} - Acc^*_{source}$.

B. Experimental Results

1) ShieldFL's Performance in Non-IID Settings: In the non-IID data setting, each user only owns 1-class non-IID data that belongs to a single class. TABLE V measures the performance of ShieldFL against both targeted and untargeted attacks. In targeted attacks, we address the the variation of Acc_{source}, Acc^*_{source} and AI_{source} between ShieldFL and the baseline. In untargeted attacks, we observe the variation of Acc, Acc^* and AI between ShieldFL and the baseline.

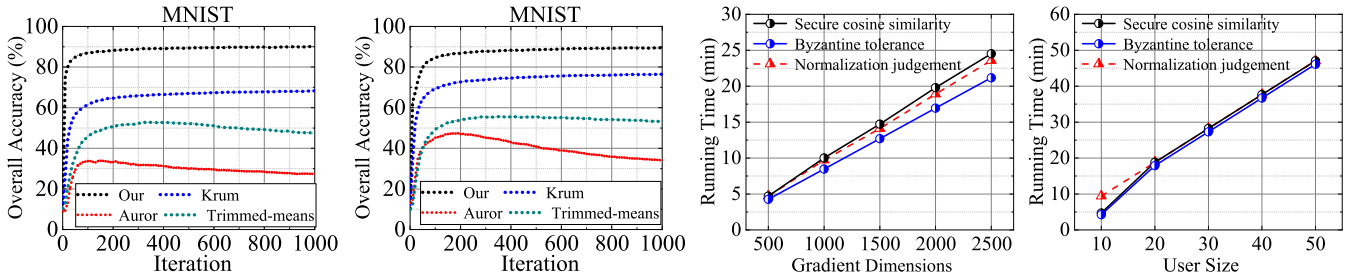
a) ShieldFL's performance against targeted attacks: Figs. 7(a)-(c) present the accuracy trajectories with $|\mathcal{U}| \in [10, 23, 50]$, where $Att_{ratio} = 50\%$. As observed in Figs. 7(a)-(c), ShieldFL has the significant improvement of Acc_{source} compared with the baseline. Besides, ShieldFL achieves stable Acc, Acc_{source} and maintains low ASR. To resist targeted attacks, ShieldFL reaches $Acc > 90\%$ and $Acc_{source} > 95\%$ over three datasets.

To evaluate the impact of Att_{ratio} in ShieldFL, Figs. 7(d)-(e) depict that Acc varies with $Att_{ratio} \in [10\%, 30\%, 50\%]$. It is observed that ShieldFL maintains a stable accuracy Acc. In ShieldFL, Acc is not affected by different levels of Att_{ratio}. Even in the case of the theoretic upper bound $Att_{ratio} = 50\%$, ShieldFL can still reach $Acc = 96.3\%$ for MNIST data, and $Acc = 95.5\%$ for KDDCUP data. The reason is that the Byzantine-tolerance aggregation empowers ShieldFL with strong robustness.

b) ShieldFL's performance against untargeted attacks: Compared with the baseline, each line on Figs. 7(f)(g) represents AI of ShieldFL under three different attack ratios $Att_{ratio} \in [10\%, 30\%, 50\%]$. Fig. 7(f) plots that ShieldFL can improve $10\% - 20\%$ test accuracy over the MNIST dataset compared with the baseline. Fig. 7(g) shows that AI remains stable with respect to $Att_{ratio} \in [10\%, 30\%, 50\%]$ over the KDDCup dataset. Fig. 7(h) demonstrates that ShieldFL has significantly improved the performance over Amazon data compared with the baseline when $Att_{ratio} = 50\%$.

2) ShieldFL's Performance in IID Settings: We also test ShieldFL in the IID setting, where training data are evenly distributed across all users. All results are measured in TABLE VI.

a) ShieldFL's performance against targeted attacks: To investigate the impact of targeted model poisoning attacks to



(a) Non-IID setting, $Att_{ratio} = 50\%$, targeted attack. (b) Non-IID setting, $Att_{ratio} = 50\%$, untargeted attack. (c) $n = 10$. (d) $m = 1,000$.

Fig. 9. Comparative analysis and efficiency analysis.

TABLE VI

PERFORMANCE COMPARISON IN THE IID SETTING ($Att_{ratio} = 50\%$)

Datasets	Attack	Baseline		ShieldFL			
		Acc^*_{source}	Acc^*	Acc_{source}	Acc	AI_{source}	AI
MNIST	Targeted	10.3%	—	95.6%	90.1%	85.3%	—
	Untargeted	—	9.9%	—	90.2%	—	80.3%
KDDCUP	Targeted	51.3%	—	99.8%	98.9%	48.5%	—
	Untargeted	—	51.3%	—	98.9%	—	47.6%
Amazon	Targeted	45.6%	—	88.5%	77.1%	42.9%	—
	Untargeted	—	22.9%	—	84.6%	—	61.7%

TABLE VII

ACCURACY COMPARISON BETWEEN SHIELDFL AND EXISTING SCHEMES

Setting	Attack	[11]	[13]	[21]	[26]	[17]	[24]	ShieldFL
Non-IID	Targeted	52.7%	71.4%	32.5%	46.4%	89.7%	37.8%	**90.1%**
	Untargeted	53.4%	78.3%	49.8%	57.4%	89.5%	43.4%	**89.4%**
IID	Targeted	82.3%	**90.7%**	89.2%	88.2%	74.5%	75.1%	90.3%
	Untargeted	85.7%	**90.4%**	87.8%	81.7%	70.8%	69.7%	90.2%

Notes. $Att_{ratio} = 50\%$, MNIST data, 500 training iterations.

ShieldFL, we plot the accuracy trajectories of ShieldFL and the baseline in Figs. 8(a-c). Here, we evaluate ShieldFL in the worst case as $Att_{ratio} = 50\%$. ShieldFL always demonstrates $Acc > 80\%$ and $ASR < 20\%$ against targeted attacks.

b) ShieldFL's performance against untargeted attacks: We test Acc and ASR of ShieldFL over three datasets against untargeted model poisoning attacks. As depicted in Fig. 8(d-f), we evaluate ShieldFL in terms of Acc and ASR and compare it to Acc^* in the baseline. Compared with Acc^* achieved by the baseline, ShieldFL accomplishes a significant accuracy improvement over three datasets.

3) Comparative Analysis: We compare ShieldFL with other existing schemes including Trimmed-means [11], Krum [13], Auror [21], PEFL [26], Sybils [17] and FL-trust [24]. TABLE VII reports the accuracy comparison in non-IID and IID data settings. It can be observed that ShieldFL consistently delivers better accuracy than any existing defense scheme in the non-IID setting. Figs. 9(a)(b) further depict the contrast of Acc against targeted attacks and untargeted attacks. In these figures, we set $|\mathcal{U}^*| = 5$ and $|\mathcal{U}| = 10$. We observe that ShieldFL performs better than other existing schemes. Due to the fact that the trimmed-means method [11] relies on Att_{ratio}, the scheme [11] performs poorly with $Acc < 60\%$ when $Att_{ratio} = 50\%$. All Krum [13], Auror [21] and PEFL [26] schemes are prone to misjudge local benign gradients trained on non-IID data since a poisonous gradient can hardly be distinguished from benign gradients with divergence. In the IID setting, ShieldFL's accuracy still remains high, which is close to the best performer Krum [13]. Compared with ShieldFL, Sybils [17] has a significant accuracy degradation in the IID setting. This is because Sybils [17] considers the two most similar gradients as outliers, which can resist model poisoning attacks in non-IID settings as poisonous gradients have the same attack objective. However, due to the similarity of benign gradients trained with IID data, the defense strategy in Sybils [17] easily causes misidentification in the IID setting. To evaluate FL-trust [24], we collect the validation data by randomly sampling from $\mathcal{U}_1$ and $\mathcal{U}_2$'s local data. However, FL-trust performs poorly in both IID and non-IID settings. The above experiments confirm that ShieldFL is better than these existing defense schemes in both non-IID and IID settings, thereby offering robustness as claimed.

4) Efficiency Performance in ShieldFL: We test the running time of ShieldFL. As shown in Fig.9(c), the running time is dominated by the dimension of gradients m that grows linearly with the value of m. The reason is that more encrypted elements are involved in SecCos with the increase of the dimension m of gradients, which requires more secure computations. The secure computation in SecCos is based on the two-trapdoor HE cryptosystem. For the 80-bit security level, we test the running time of each algorithm in two-trapdoor HE, where Enc costs 6.74 ms, PartDec costs 14.39 ms, FullDec costs 0.01 ms. Compared with the running time 6.88 ms of Dec in the single-key HE, the running time of decryption algorithms PartDec and FullDec is increasing, but within acceptable limits. As plotted in Fig.9(d), the running time grows linearly with the number of users n. With $n = 10$ and $m = 1,000$, the privacy-preserving defense strategy costs 18.4 min, in which the normalization judgement process costs 9.4 min, secure cosine similarity costs 4.7 min, and Byzantine-tolerance aggregation process costs 4.3 min. These schemes [11], [13], [21] are performed over plaintexts without privacy guarantees, whose running time is below < 1 min. Compared with [11], [13], [21], ShieldFL imposes an additional cost for privacy protection within an acceptable range.

a) Discussion: We emphasize that our experiments are conducted on an average six-core PC with Intel i7-9750H processor 4.50GHz and 16GB RAM. We expect that the

running time performance of ShieldFL can be significantly improved if more powerful computers (*i.e.,* cloud servers and high-performance computers) are used to implement the two servers $\mathcal{S}_1$ and $\mathcal{S}_2$ in practice. In such case, a larger number of users can be efficiently supported using ShieldFL. Beyond the scope of the current work, we still address the issue of the mass increase of users. We recognize that the efficiency of ShieldFL is not highly scalable to the number of users n. With the growth of users, there are more encrypted local gradients submitted for the privacy-preserving defense strategy, which causes the increase of communication and computational overheads. One potential approach to increase the scalability of ShieldFL is to randomly select a subset of users to perform local training and model updates in each iteration of PPFL. Such approach has been explored in existing federated learning schemes [43], which reduces the overheads caused by the increase of users without sacrificing the accuracy of the federated model significantly.

VIII. Conclusion

This paper proposes ShieldFL, a privacy-preserving defense strategy against model poisoning attacks in PPFL. In ShieldFL, the proposed defense strategy is based on the secure cosine similarity over encrypted local gradients. It provides a viable solution to detect encrypted malicious gradients and thus thwart encrypted model poisoning attacks in PPFL. ShieldFL leverages two-trapdoor homomorphic encryption to guarantee data privacy in PPFL. It relies on the Byzantine-tolerance aggregation mechanism to maintain its high accuracy in both non-IID and IID data settings.

References

[1] W. Zheng, R. A. Popa, J. E. Gonzalez, and I. Stoica, "Helen: Maliciously secure coopetitive learning for linear models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 724–738.

[2] K. Bonawitz *et al.*, "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2017, pp. 1175–1191.

[3] P. Xie, B. Wu, and G. Sun, "BAYHENN: Combining Bayesian deep learning and homomorphic encryption for secure DNN inference," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, Aug. 2019, pp. 4831–4837.

[4] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 3–18.

[5] R. Shokri and V. Shmatikov, "Privacy-preserving deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Sep. 2015, pp. 1310–1321.

[6] A. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 634–643.

[7] L. Melis, C. Song, E. D. Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 691–706.

[8] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2020, pp. 2938–2948.

[9] X. Cao, J. Jia, and N. Z. Gong, "Provably secure federated learning against malicious clients," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2021, vol. 35, no. 8, pp. 6885–6893.

[10] B. Hou *et al.*, "Mitigating the backdoor attack by federated filters for industrial IoT applications," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3562–3571, May 2022.

[11] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in *Proc. USENIX Secur. Symp.*, 2020, pp. 1605–1622.

[12] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 5650–5659.

[13] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 1–11.

[14] H. Oh and Y. Lee, "Exploring image reconstruction attack in deep learning computation offloading," in *Proc. Int. Workshop Deep Learn. Mobile Syst. Appl. (MobiSys)*, 2019, pp. 19–24.

[15] Z. Wang, M. Song, Z. Zhang, Y. Song, Q. Wang, and H. Qi, "Beyond inferring class representatives: User-level privacy leakage from federated learning," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Apr. 2019, pp. 2512–2520.

[16] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 739–753.

[17] C. Fung, C. J. Yoon, and I. Beschastnikh, "The limitations of federated learning in Sybil settings," in *Proc. Int. Symp. Res. Attacks, Intrusions Defenses (RAID)*, 2020, pp. 301–316.

[18] C. Xie, K. Huang, P.-Y. Chen, and B. Li, "DBA: Distributed backdoor attacks against federated learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019, pp. 1–19.

[19] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," in *Proc. Eur. Symp. Res. Comput. Secur. (ESORICS)*. Springer, 2020, pp. 480–501.

[20] R. Guerraoui *et al.*, "The hidden vulnerability of distributed learning in Byzantium," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 3521–3530.

[21] S. Shen, S. Tople, and P. Saxena, "AUROR: Defending against poisoning attacks in collaborative deep learning systems," in *Proc. Annu. Conf. Comput. Secur. Appl. (ACSAC)*, Dec. 2016, pp. 508–519.

[22] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," 2018, *arXiv:1806.00582*.

[23] H. Wang, Z. Kaplan, D. Niu, and B. Li, "Optimizing federated learning on non-IID data with reinforcement learning," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Jul. 2020, pp. 1698–1707.

[24] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2021.

[25] M. Goddard, "The EU general data protection regulation (GDPR): European regulation that has a global impact," *Int. J. Market Res.*, vol. 59, no. 6, pp. 703–705, Nov. 2017.

[26] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, and R. Lu, "Privacy-enhanced federated learning against poisoning adversaries," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4574–4588, 2021.

[27] X. Liu, R. H. Deng, K.-K. R. Choo, and J. Weng, "An efficient privacy-preserving outsourced calculation toolkit with multiple keys," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 11, pp. 2401–2414, Nov. 2016.

[28] X. Liu, K. K. R. Choo, R. H. Deng, R. Lu, and J. Weng, "Efficient and privacy-preserving outsourced calculation of rational numbers," *IEEE Trans. Depend. Sec. Comput.*, vol. 15, no. 1, pp. 27–39, Jan./Feb. 2018.

[29] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 19–38.

[30] D. Yang, B. Xu, B. Yang, and J. Wang, "Secure cosine similarity computation with malicious adversaries," Tech. Rep., 2013.

[31] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li, "Manipulating machine learning: Poisoning attacks and countermeasures for regression learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2018, pp. 19–35.

[32] M. Song *et al.*, "Analyzing user-level privacy attack against federated learning," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2430–2444, 2020.

[33] D. Pasquini, G. Ateniese, and M. Bernaschi, "Unleashing the tiger: Inference attacks on split learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Nov. 2021, pp. 2113–2129.

[34] P. Paillier and D. Pointcheval, "Efficient public-key cryptosystems provably secure against active adversaries," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 1999, pp. 165–179.

[35] M. Keller, "MP-SPDZ: A versatile framework for multi-party computation," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2020, pp. 1575–1590.

[36] D. Bogdanov, S. Laur, and J. Willemson, "Sharemind: A framework for fast privacy-preserving computations," in *Proc. Eur. Symp. Res. Comput. Secur. (ESORICS)*. Springer, 2008, pp. 192–206.
[37] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020, pp. 1–26.
[38] D. Catalano, R. Gennaro, and N. Howgrave-Graham, "The bit security of Paillier's encryption scheme and its applications," in *Proc. Int. Conf. Theory Appl. Cryptograph. Techn. (EUROCRYPT)*. Springer, 2001, pp. 229–243.
[39] Y. LeCun, C. Cortes, and C. J. Burges. (1998). *MNIST Database*. [Online]. Available: http://yann.lecun.com/exdb/mnist
[40] UCI KDD. (1999). *KDD Cup 1999 Data*. [Online]. Available: http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html
[41] Amazon. (2003). *Amazon Networks*. [Online]. Available: http://snap.stanford.edu/data/amazon/
[42] Y. Huang *et al.*, "Personalized cross-silo federated learning on non-IID data," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 9, pp. 7865–7873.
[43] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist. (AISTATS)*, 2017, pp. 1273–1282.

Zhuoran Ma is currently pursuing the Ph.D. degree with the Department of Cyber Engineering, Xidian University. Her current research interests include data security and secure computation outsourcing.

Jianfeng Ma (Member, IEEE) received the Ph.D. degree in computer software and telecommunication engineering from Xidian University, Xi'an, China, in 1995. From 1999 to 2001, he was a Research Fellow with Nanyang Technological University of Singapore. He is currently a Professor and a Ph.D. Supervisor with the Department of Computer Science and Technology, Xidian University. He is also the Director of the Shaanxi Key Laboratory of Network and System Security. His current research interests include information and network security, wireless and mobile computing systems, and computer networks.

Yinbin Miao (Member, IEEE) received the B.E. degree from the Department of Telecommunication Engineering, Jilin University, Changchun, China, in 2011, and the Ph.D. degree from the Department of Telecommunication Engineering, Xidian University, Xi'an, China, in 2016. He is currently a Lecturer with the Department of Cyber Engineering, Xidian University. His research interests include information security and applied cryptography.

Yingjiu Li (Member, IEEE) is currently a Ripple Professor with the Computer and Information Science Department, University of Oregon. He has published over 140 technical papers in international conferences and journals, and served in the program committees for over 80 international conferences and workshops, including top-tier cybersecurity conferences and journals. His research interests include the IoT security and privacy, mobile and system security, applied cryptography and cloud security, and data application security and privacy.

Robert H. Deng (Fellow, IEEE) has been a Professor with the School of Information Systems, Singapore Management University, since 2004. His research interests include data security and privacy, multimedia security, and network and system security. He received the University Outstanding Researcher Award from the National University of Singapore in 1999 and the Lee Kuan Yew Fellow for Research Excellence from Singapore Management University in 2006. He was named as a Community Service Star and Showcased Senior Information Security Professional by $(ISC)^2$ under its Asia-Pacific Information Security Leadership Achievements Program in 2010. He is the Co-Chair of the Steering Committee of the ACM Symposium on Information, Computer and Communications Security. He was an Associate Editor of the IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY from 2009 to 2012. He is currently an Associate Editor of the IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING and *Security and Communication Networks* (John Wiley).